UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO

INSTITUTO DE QUÍMICA

DEPARTAMENTO DE QUÍMICA ANALÍTICA

INTRODUÇÃO AO SOFTWARE R

Prof. Dr. Aderval S. Luna

Prof. Dr. Igor Campos de Almeida Lima

1ª versão (2018/2)

I. INTRODUÇÃO

O R é um software de fonte aberta, um pacote estatístico de computação numérica de multi-plataforma de alto nível. Possui uma linguagem de programação numericamente orientada para o desenvolvimento de um software estatístico e análise de dados. A linguagem de programação **R** é baseada na programação de linguagem **S**, que foi desenvolvida nos Laboratórios Bell. O software **R** não foi desenvolvido como um clone do Matlab, e, portanto, tem uma sintaxe diferente dos pacotes de análise numérica anteriores. Este software faz parte do projeto GNU e pode ser baixado a partir de www.r-project.org. Pacotes adicionais desenvolvidos pela comunidade de usuários do **R** podem ser baixados do CRAN (The Comprehensive R Archive Network) em cran.r-project.org.

1. Recursos Estatísticos do pacote R

O R fornece uma grande variedade de técnicas estatísticas, as quais se incluem a modelagem linear e não linear, os testes clássicos de estatística, análise de séries temporais, métodos de classificação, análise de aglomerados e outros. O software R é facilmente extensível através de funções e a comunidade do R é conhecida por suas contribuições ativas em termos de pacotes. Existem algumas diferenças importantes, mas muito dos códigos escritos para S funcionam no R de forma inalterada. Muitas das funções padrão do R são escritos em R, o que torna mais fácil para os usuários seguir as escolhas dos algoritmos feitas. Usuários avançados podem escrever código em C ou Java para manipular objetos diretamente no R. O R é altamente extensível por meio da utilização de pacotes enviados pelos usuários para funções específicas ou áreas específicas de estudo. Outro ponto forte do R é a produção de gráficos com qualidade de publicação, incluindo símbolos matemáticos. Gráficos dinâmicos e interativos estão disponíveis através de pacotes adicionais. O R tem o seu próprio formato de documentação semelhante ao LaTex, que é usado para documentação, tanto on-line quanto em numerosos formatos ou em papel. Apesar do R ser usado principalmente por estatísticos, outros profissionais que necessitam de um ambiente para o cálculo estatístico e de ferramentas de cálculo com um desempenho comparável ao GNU Octave ou MATLAB podem ser usuários.

1. GUIA DE INSTALAÇÃO DO SOFTWARE *R* E DA INTERFACE *RSTUDIO*

OBS: A interface gráfica *RStudio* só funciona se o software *R* estiver sido previamente instalado.

Download e instalação do software R:

- Vá para o endereço: http://www.r-project.org
- Clique em CRAN (*Comprehensive R Archive Network*).
- Escolha o *espelho* mais próximo (No caso da UERJ, escolha FIOCRUZ)
- Clique em "Download and Install R", "Windows".
- Após instalar, vá em *Iniciar*/*Todos os Programas*/*R* ou clique no ícone na área de trabalho.

Download e instalação da interface gráfica RStudio:

- Vá para o endereço: http://www.rstudio.com/products/rstudio/download/
- Selecione a plataforma
- Faça o download do programa e instale-a.

2. CONHECENDO O RSTUDIO

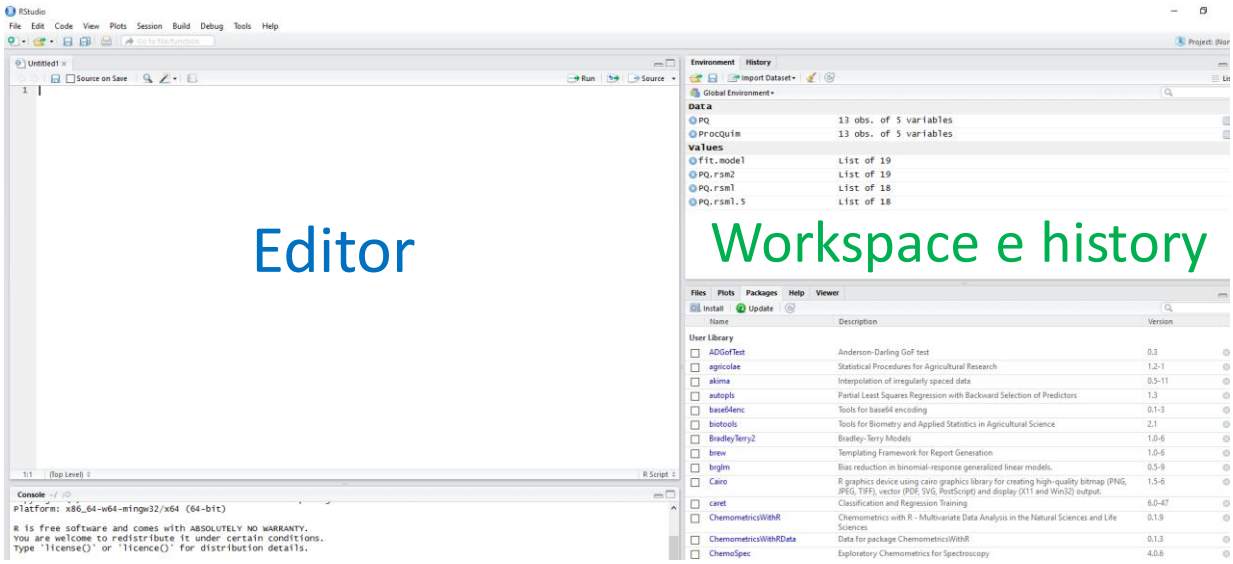
O *R Studio* possui quatro janelas.

A janela do canto superior esquerdo é um editor (como o Tinn-R),

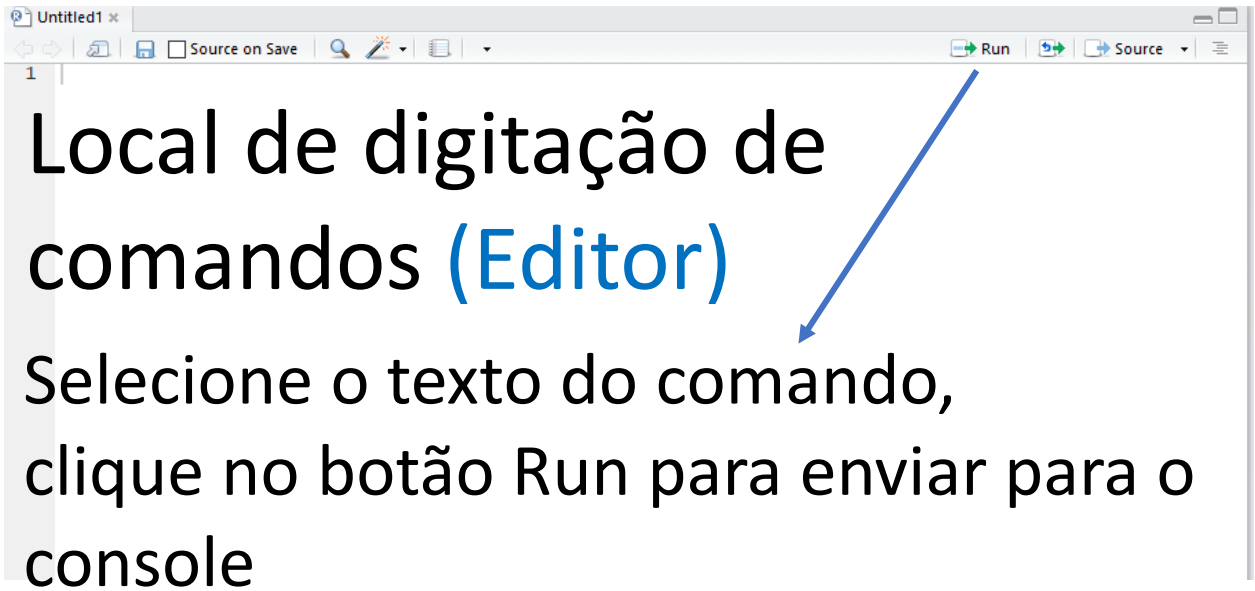
A do canto inferior esquerdo é o *console* do R,

A do canto superior direito ficam o *workspace* e o *history* e

A janela do canto inferior esquerdo apresenta o *help*, os pacotes e os gráficos.



Help, pacotes e gráficos

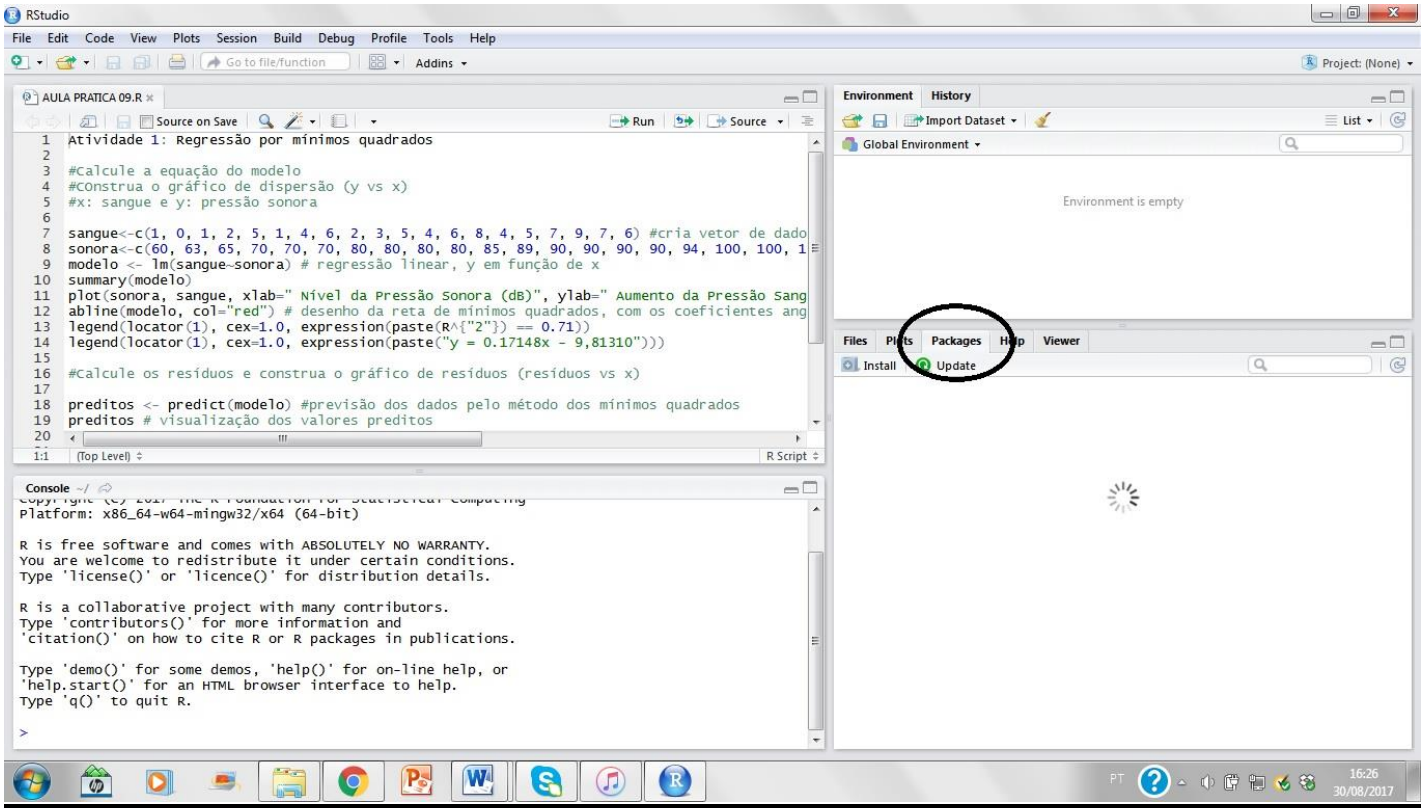


Local de digitação de comandos (Editor)

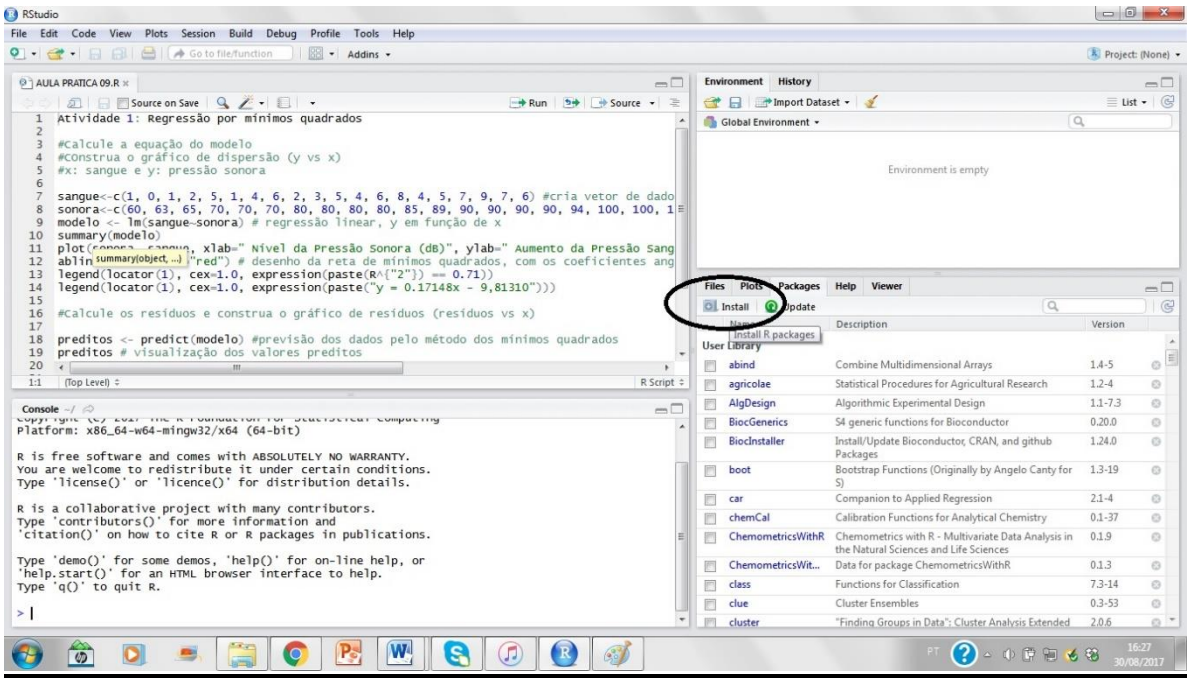
Selecione o texto do comando, clique no botão Run para enviar para o console

INSTALAÇÃO DE PACOTES NO RSTUDIO

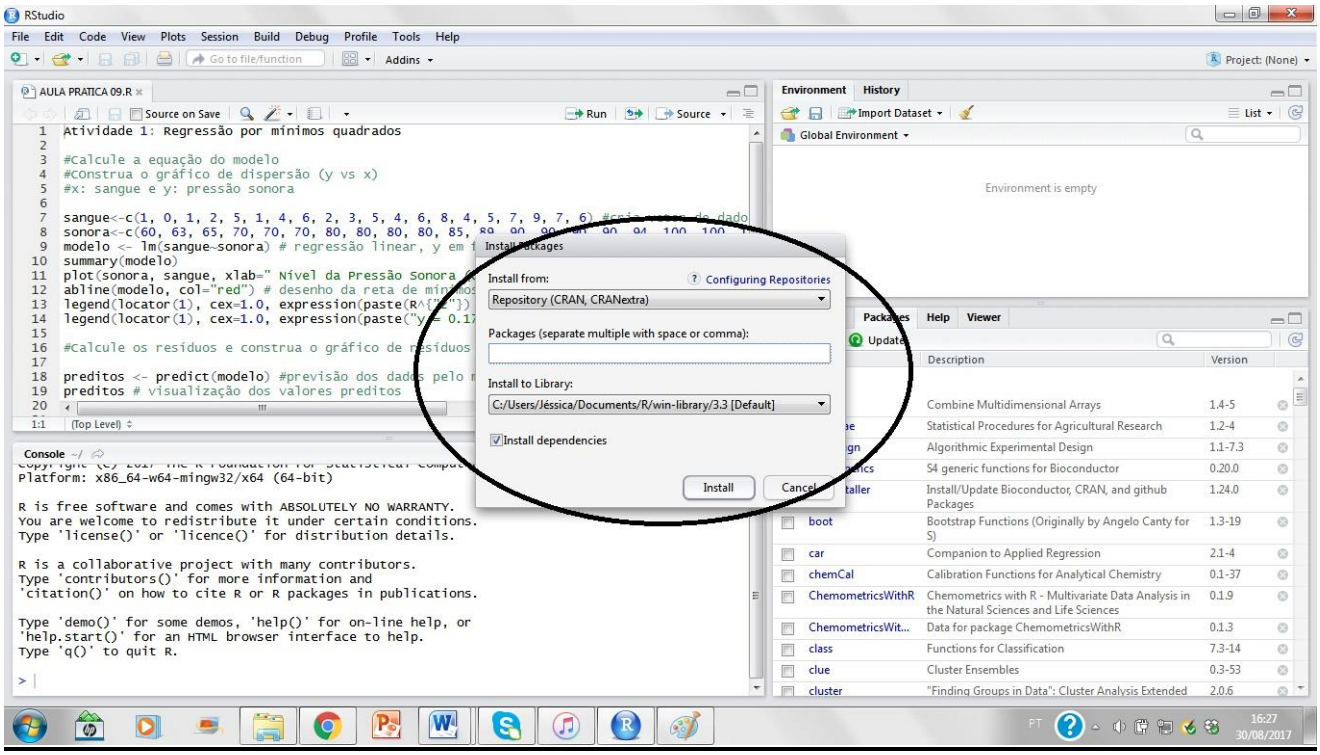
1. CLICAR EM PACKAGES



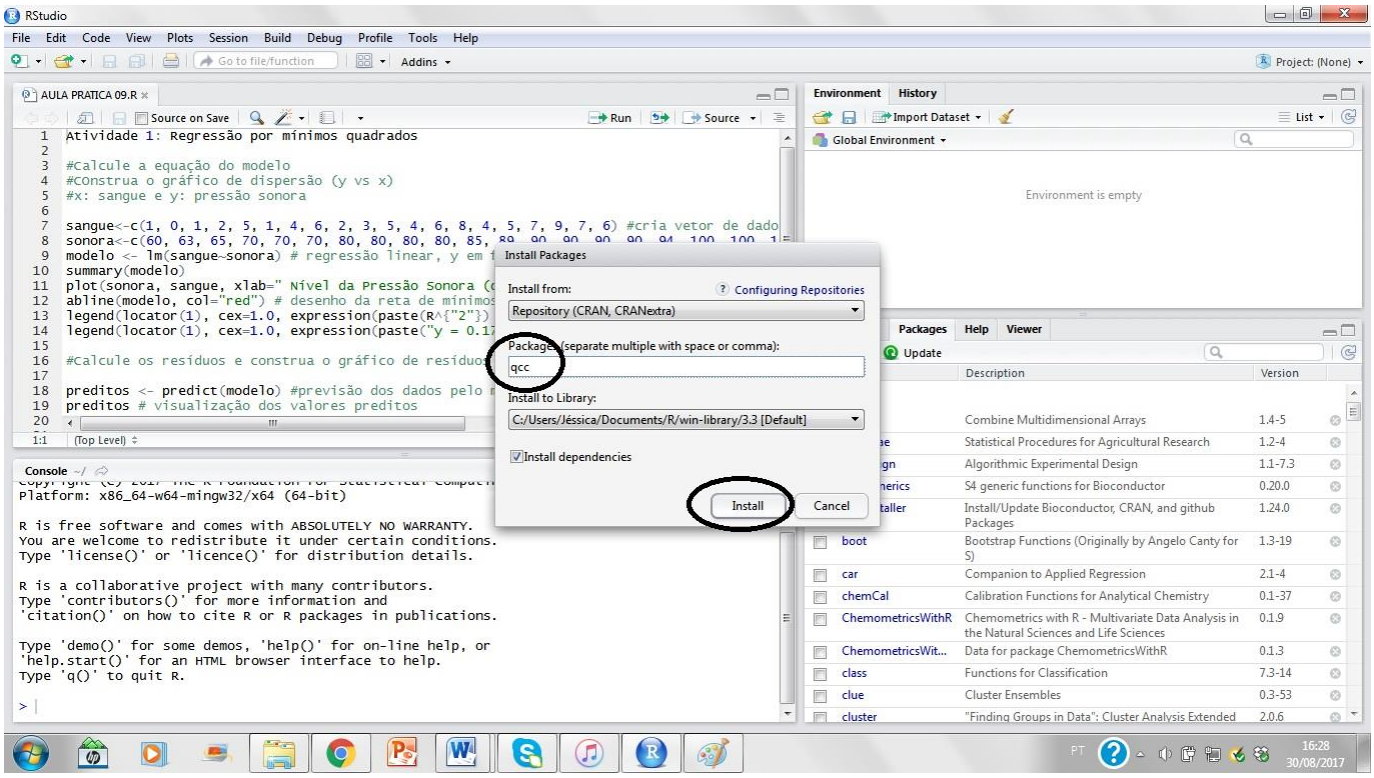
2. CLICAR EM INSTALL



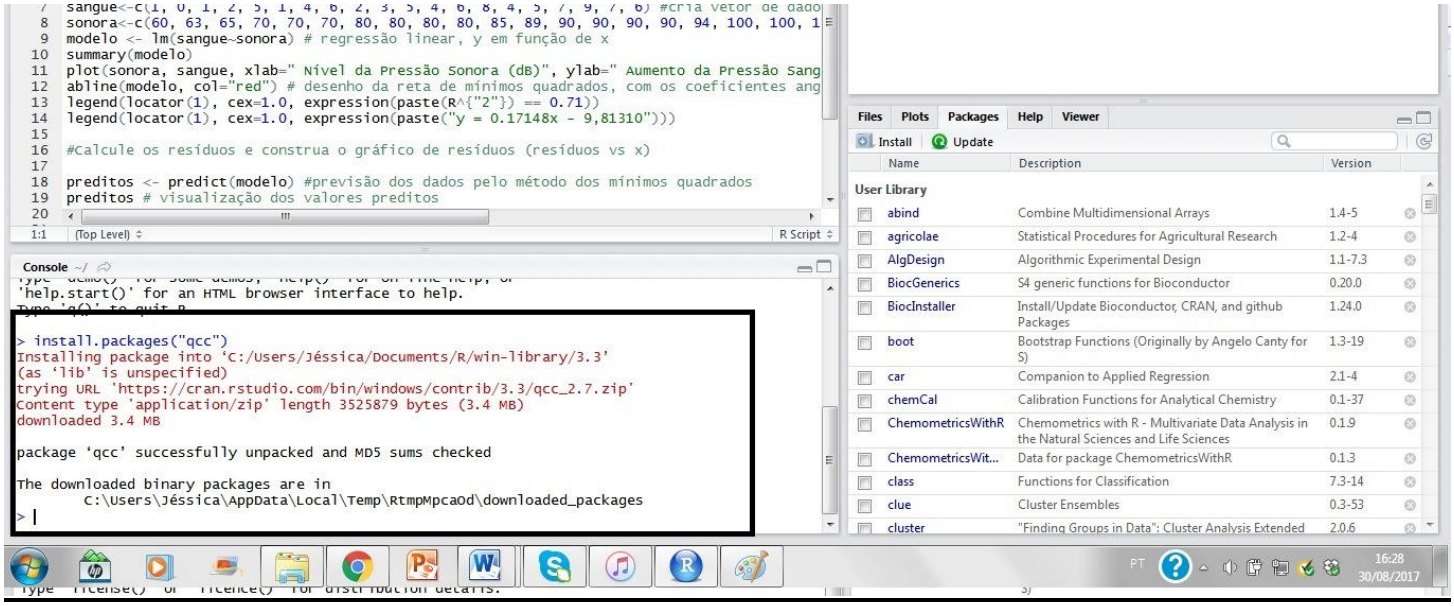
3. IRÁ ABRIR UMA JANELA COMO A SEGUIR



4. DIGITE O NOME DO PACOTE A SER INSTALADO E CLIQUE EM INSTALL

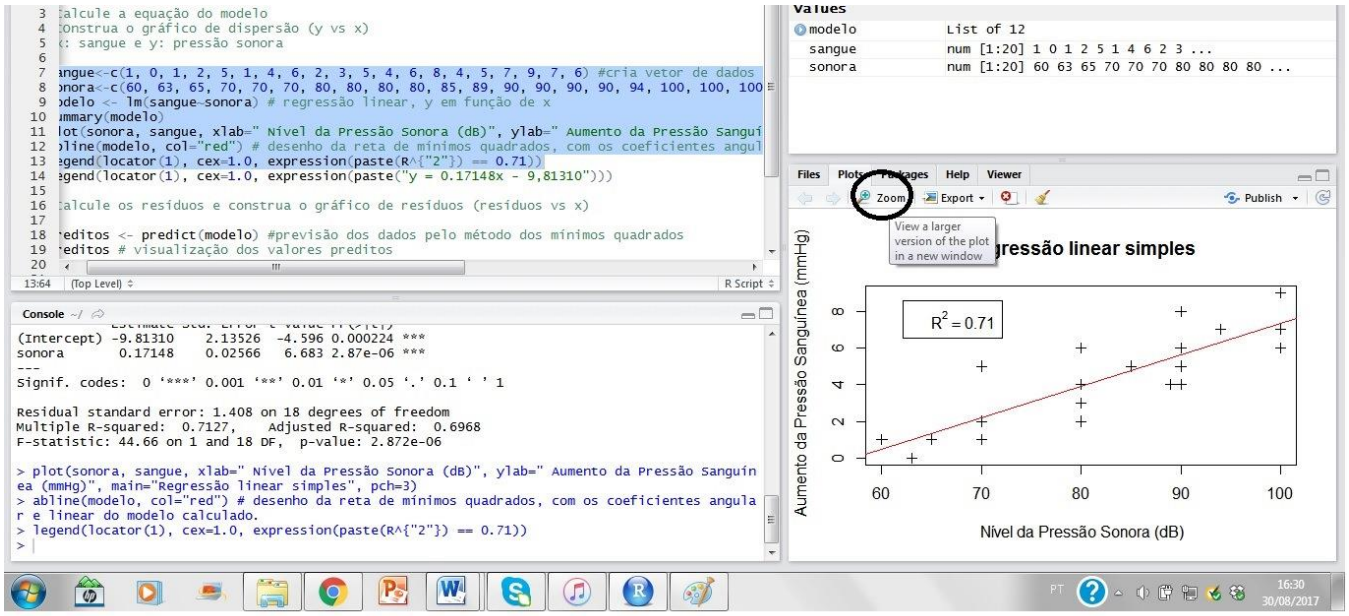


5. NO CONSOLE IRÁ APARECER O DIRETÓRIO NO QUAL O PACOTE FOI INSTALADO

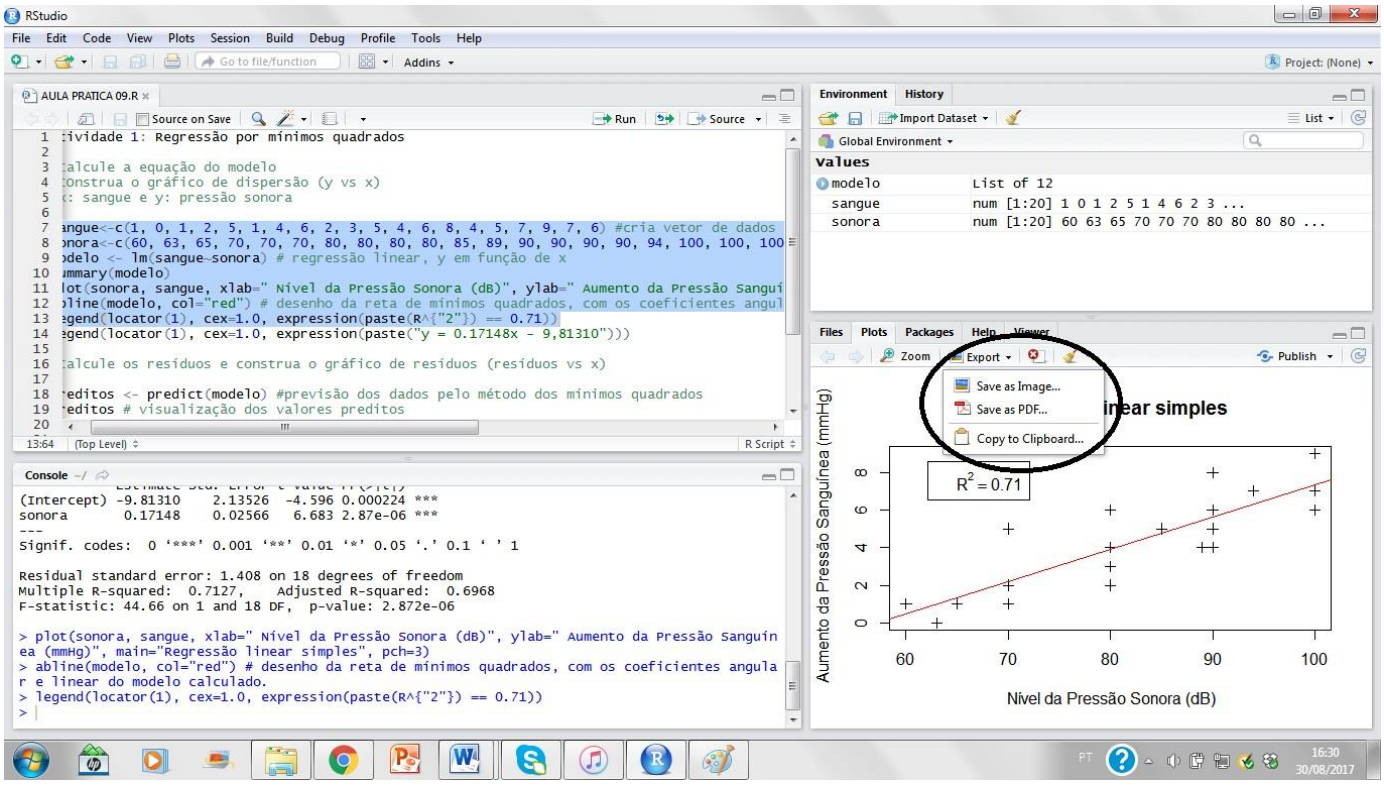


3. VISUALIZANDO E SALVANDO IMAGENS NO RSTUDIO

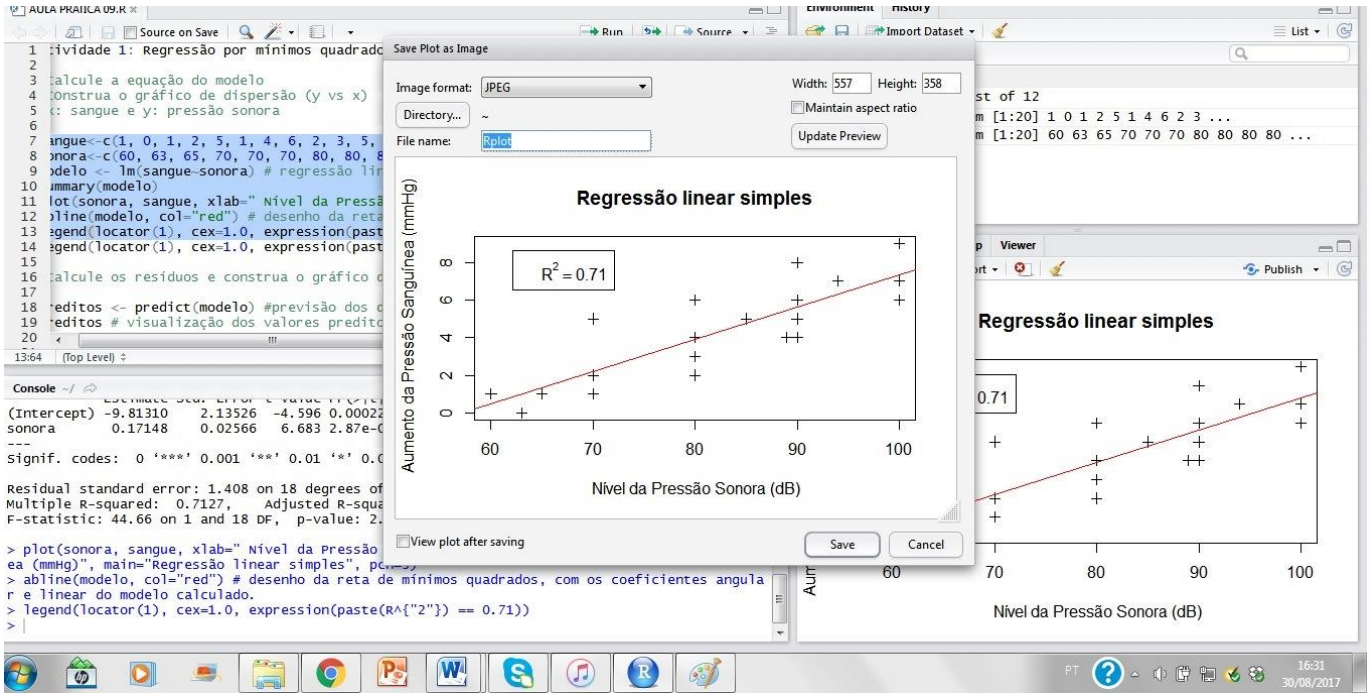
1. CLICAR EM ZOOM, ISTO FARÁ A IMAGEM ABRIR EM OUTRA JANELA



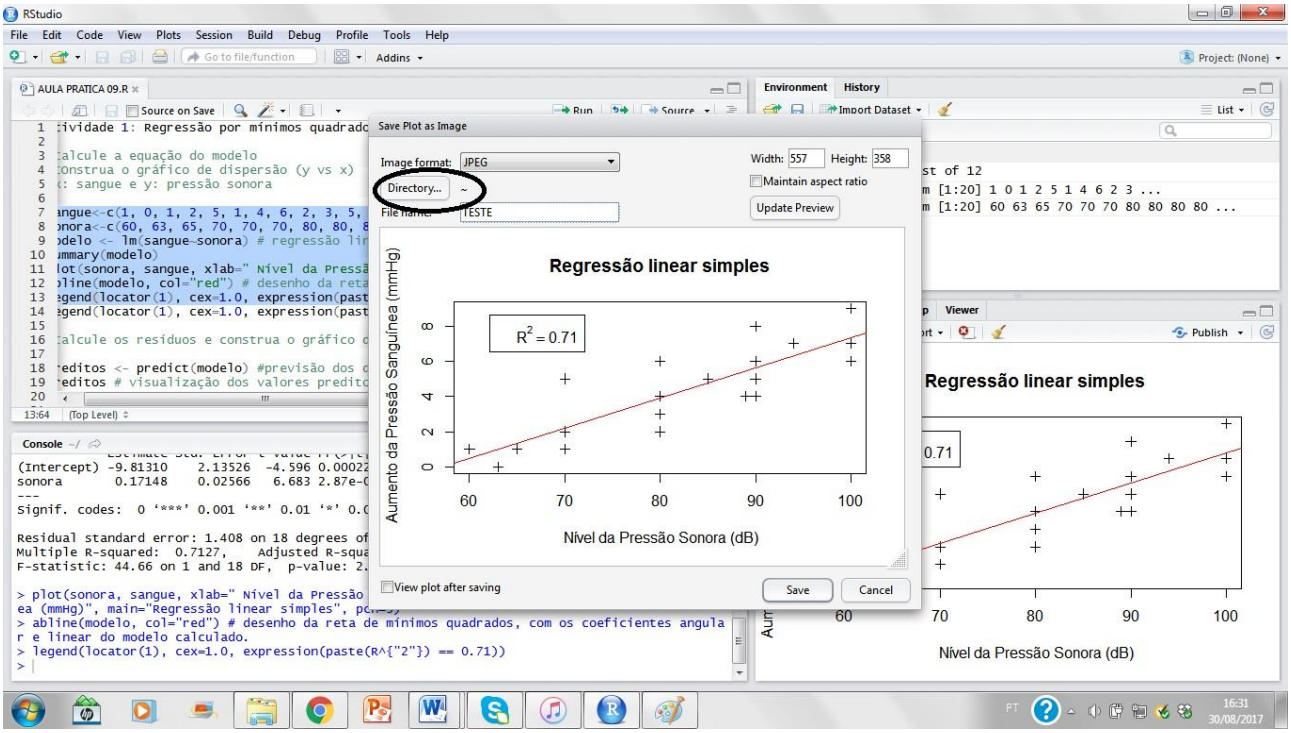
2. PARA SALVAR A IMAGEM, CLICAR EM EXPORT E ESCOLHER SE SALVA COMO IMAGEM OU ARQUIVO PDF



3. IRÁ ABRIR UMA JANELA COMO A SEGUIR

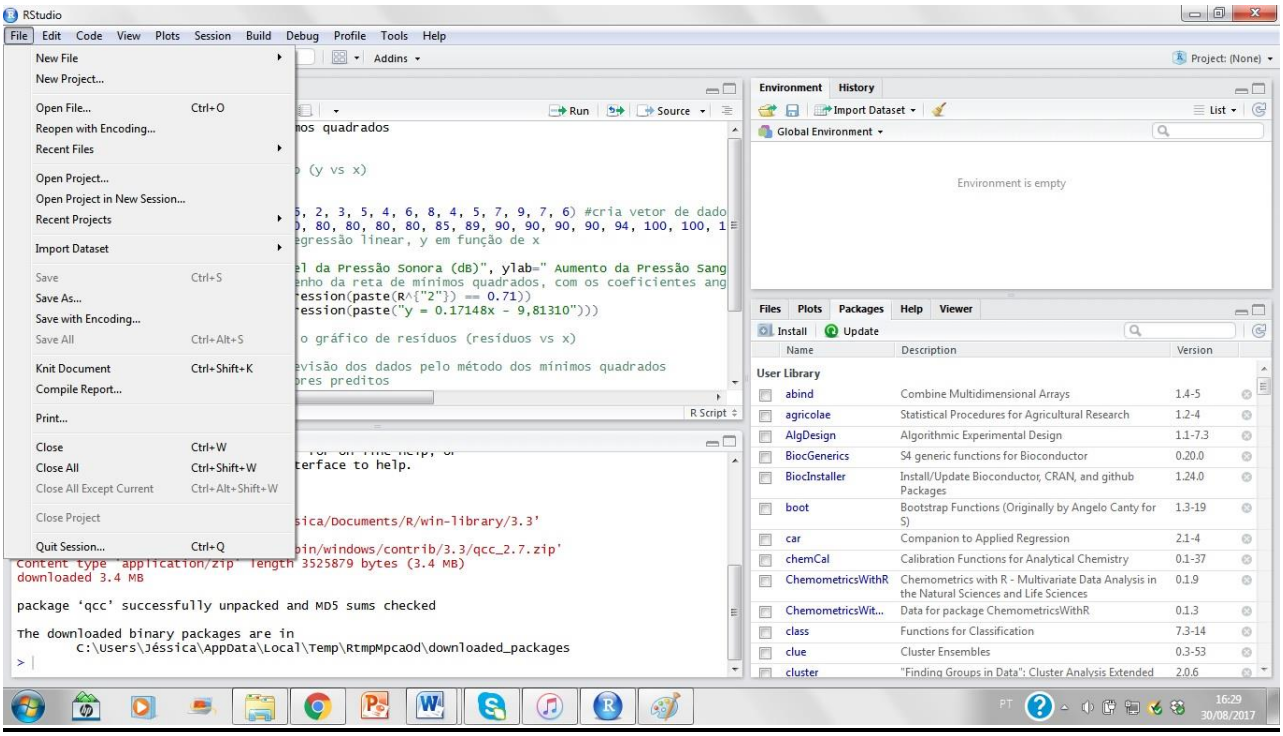


4. ESCOLHER O DIRETÓRIO E DAR NOME AO ARQUIVO E CLICAR EM SAVE



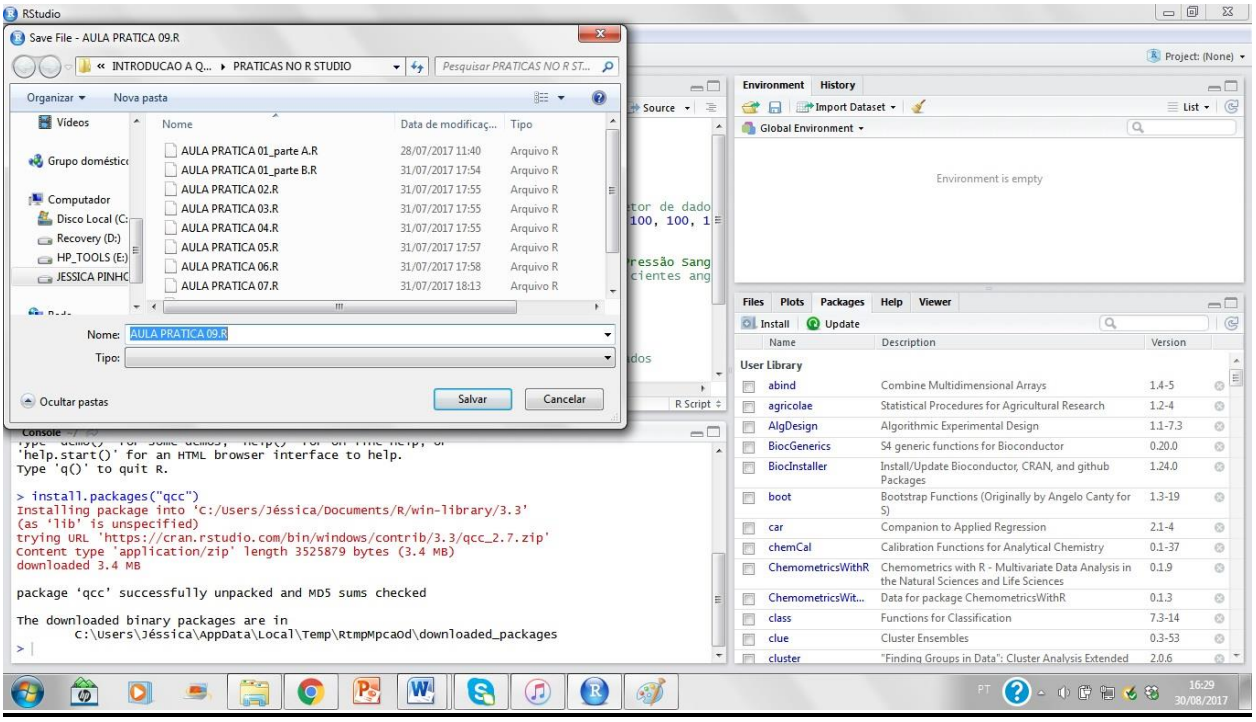
SALVANDO SCRIPT NO RSTUDIO

1. CLICAR EM FILE



2. CLICAR EM SAVE FILE AS

3. DIGITAR O NOME DO ARQUIVO A SER SALVO NA EXTENSÃO . R NO DIRETÓRIO DESEJADO



ORIENTAÇÕES ADICIONAIS NO R - STUDIO

1. VERIFICANDO E TROCANDO DIRETÓRIO NO R – STUDIO

- Verifique o diretório atual:

  getwd()

- Troque o diretório

  setwd("G:/UERJ")

- Verifique se o diretório atual é o desejado:

  getwd()

2. IMPORTAÇÃO DE ARQUIVOS

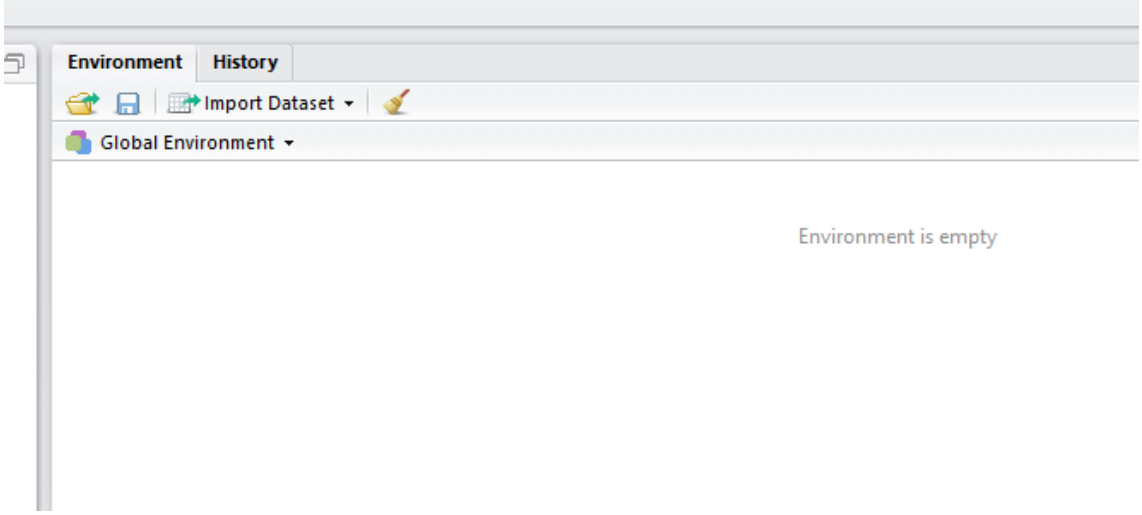
2.1 OPÇÃO A

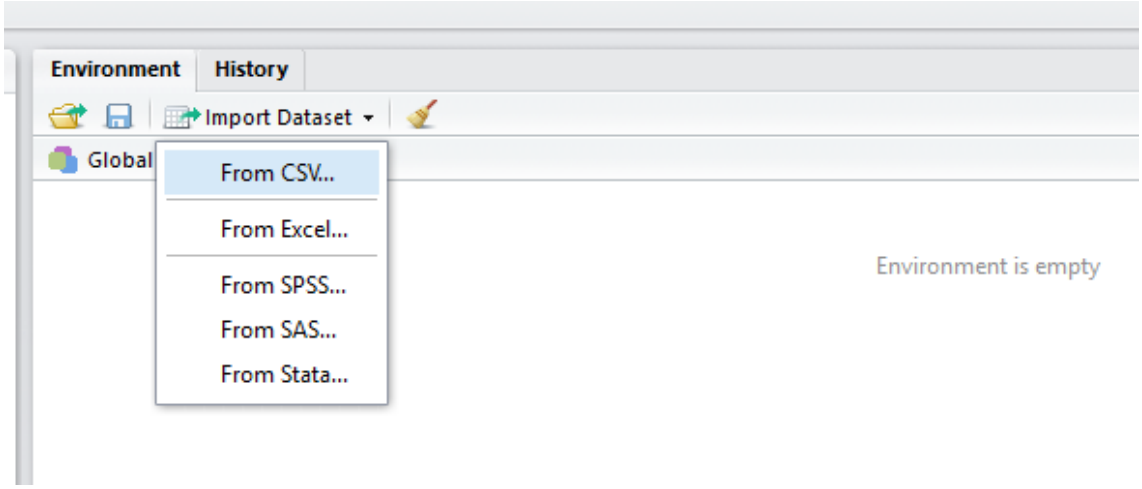
• Crie um arquivo no Excel, com dados numéricos, com duas colunas e 10 linhas, e nomes nos cabeçalhos. A seguir, salve na extensão *.CSV (separado por vírgulas). Faça a importação para o R, suando o comando abaixo.

arquivo.csv <- read.csv("C:/diretório onde o arquivo está/nome do arquivo.csv", header=TRUE, sep="", dec=".")

2.2 OPÇÃO B

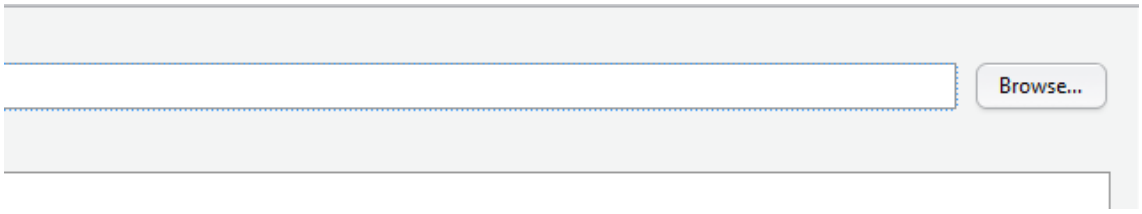
Clique em “Import Dataset”.





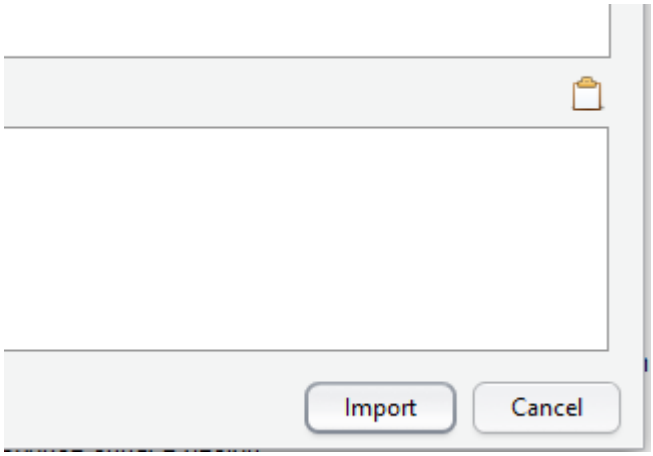
Escolha o tipo de arquivo (From CSV; From Excel; From SPSS; From SAS; From Stata), depois clique no tipo de arquivo escolhido.

Clique em “Browse” para buscar o caminho do arquivo a ser importado.



Caso a primeira linha do arquivo tenha nome ou legenda, marque “First Row as Names”. Em caso contrário, desmarque-a.

Posteriormente, clique em “Import”.



A Linha de Comando do R

O Software R é sensível às letras maiúsculas e minúsculas. As funções podem ou não requerer parâmetros que serão tratados, neste documento, como argumentos. Os argumentos devem ser informados entre os parênteses que vêm logo após o nome da função. Caso nenhum argumento seja informado, ainda assim deve-se abrir e fechar os parênteses.

Outro fato importante deste software é a forma como informamos os argumentos em uma função. Nomeiam-se os argumentos ou informa-os sem nomes, mas em ordem pré-determinada. Por exemplo, a função lm possui mais de dez argumentos que podem ser informados. No código exemplificado abaixo, são passados os dois primeiros argumentos e os demais são ignorados. O software sabe quais são os argumentos pela ordem dos mesmos.

```
> lm(y~x,cars)
```

Neste outro exemplo, a função é chamada da mesma forma, porém os argu- m entos são nomeados.

```
> lm(formula = y~x,data=cars)
```

Como um programa de linha de comando, você deve digitar na posição do cursor e depois clicar “Enter”. Neste momento, o R executa o comando. Um comando pode gerar uma saída, sendo uma mensagem de erro, dados de um data frame ou mesmo o valor de uma variável. A saída será apresentada logo abaixo do comando de execução. Por exemplo, o comando “getwd” tem como saída o diretório de trabalho.

```
> getwd()
[1] "C:/Users/user/Documents"
```

Caso você digite vários comandos separados por ponto e vírgula, o R irá executá-los simulteamente apenas após o “Enter”.

```
> getwd();b=3;print(b)
[1] "C:/Users/user/Documents"
[1] 3
```

Histórico

Você pode navegar entre os últimos comandos para reutilização através das teclas seta para cima e seta para baixo. Pode-se ainda obter uma lista dos últimos comandos digitando “history”. Esta função sem argumentos lista em nova janela os 26 últimos comandos digitados. Caso você queira limitar o número de

comandos na saída através do argumento “max.show”. Neste exemplo, o R lista os últimos cinco comandos executados:

```
> history(max.show=5)
```

Como obter ajuda no R

Como toda linguagem de programação, o R possui vasto material na Web: Fóruns, tutorial, documentos em sites diversos e no CRAN. Além disso, a instalação padrão do R tem uma série de documentos que podem ser acessados através do Menu RGui. No entanto, a melhor forma de se obter ajuda de funções é através da linha de comando, principalmente devido à velocidade e precisão.

A ajuda na linha de comando carrega um arquivo HTML instalado localmente no seu computador. O comando help.start abre a página inicial da ajuda.

```
> help.start()
If nothing happens, you should open
‘http://127.0.0.1:20878/doc/html/index.html’ yourself
```

Para obter a ajuda sobre uma função específica, utilize a função “help”, informando como parâmetro o comando que se quer consultar:

```
> help("search")
```

Obtém-se o mesmo efeito através de um ponto de interrogação seguido do comando:

```
> ?search
```

Caso você lembre da função, mas se tiver dúvida sobre os argumentos da mesma, pode-se usar a função “args”. No exemplo abaixo, consultamos os argumentos da função “sd” usada para calcular o desvio padrão:

```
> args("sd")
function (x, na.rm = FALSE)
NULL
```

Fechar o R

Você pode fechar o R usando a janela principal do RGui ou através da função q(), do inglês quit. Independente da forma, uma caixa de diálogo será exibida com a mensagem “`Save workspace image to ~/.RData? [y/n]:`” . O R irá salvar o estado da sua seção de trabalho atual através de um arquivo com

extensão como “rdata” gravado no seu diretório de trabalho. Na próxima vez que o R for iniciado, ele automaticamente carregará sua seção através deste arquivo.

Tipos de Dados

Os tipos mais comuns de dados no R são “character”, “numeric” e “integer”. A declaração e tipagem de dados no R são feitas implicitamente. Basta você definir a variável e atribuir à mesma, que automaticamente a variável é criada e sua classe é atribuída de acordo com o tipo de dado. Para se atribuir um dado a uma variável, pode-se usar o operador <- ou =. A primeira convenção nos permite atribuir um valor a uma variável à esquerda <- ou mesmo à direita ->.

No exemplo abaixo, declaramos uma variável de nome var e atribuímos à mesma o valor 68. Em seguida, digitamos a variável e o R exibe como saída o valor da variável.

```
> var<-68
> var
[1] 68
```

Caso seja necessário verificar que tipo foi atribuído ao objeto, utiliza-se a função “class” que retorna um texto com o tipo da classe. Neste caso, o R definiu o objeto com o tipo “numeric”.

```
> class(var)
[1] "numeric"
```

Opcionalmente, você pode testar o tipo de objeto com a função “is.numeric”. A saída é booleana, ou seja, lógica podendo assumir apenas valores “True/False”, conforme o resultado do teste:

```
> is.numeric(var)
[1] TRUE
```

Os caracteres são representados entre aspas duplas ou simples. Dessa forma, se atribuirmos dados entre aspas a uma variável, o R irá entender que estamos criando um objeto do tipo “character”.

```
> var1<-"Estatística"
> var1
[1] "Estatística"
```

A concatenação de textos é feita através da função “paste”. Observe o exemplo a seguir:

```
> a<-"Estatística usando"
> b<-"R"
> x<-paste(a,b)
> x
[1] "Estatística usando R"
```

Sabe-se que quando atribuímos um valor a uma variável, o objeto criado é do tipo numérico. Entretanto, se quisermos criar uma variável do tipo “integer”, podemos optar por duas maneiras distintas:

- Uma forma é colocando o caractere L após o valor;
- Outra forma é usando a função “as.integer”.

O código abaixo cria um objeto inteiro usando a notação L, em seguida, é criado um novo objeto usando a conversão para inteiro. Finalmente, verifica-se a classe dos objetos através da função “class”.

```
> varInt_1<-68L
> varInt_2<-as.integer(68)
> class(varInt_1);class(varInt_2)
[1] "integer"
[1] "integer"
```

Quando você cria objetos, eles ficam disponíveis no ambiente (Environment) até serem removidos. Os objetos podem ser removidos explicitamente usando a função “rm”, ou quando o R for fechado. Você pode listar os objetos disponíveis, no ambiente, através da função “objects”, como mostrado a seguir:

```
> objects()
[1] "varInt_1" "varInt_2"
```

Quando for necessário a remoção de apenas um objeto, digite o comando:

```
> varInt_1<-68L
> rm(varInt_1)
> objects()
character(0)
```

Observe que foi removido o objeto “varInt_1”. Caso exista mais de um objeto, e mpregue a função “list” para transformar o retorno da função “objects” em uma l ista como mostrado a seguir:

```
> rm(list=objects())
> objects()
character(0)
```

Valores não disponíveis

Valores não disponíveis são representados no R por NA (not available). É relevante conhecer o que são e como eles se comportam, pois a ocorrência de um resultado inesperado em uma expressão pode ser oriunda de valores não disponíveis. Para demonstrar, iremos criar um tipo NA:

```
> b<-NA
> b
[1] NA
```

Uma operação qualquer sobre um valor não disponível, resulta em um valor não disponível.

```
> b+1
[1] NA
```

Pode-se testar se um valor não está disponível usando a função “is.na”.

```
> is.na(b)
[1] TRUE
```

Fórmulas

O R permite que sejam construídas expressões que representam fórmulas utilizando-se o símbolo til (~) para separar a variável de resposta ou dependente. As variáveis independentes devem ficar à direita do til, já as variáveis dependentes são postas à esquerda do símbolo. Caso a fórmula seja constituída por mais de uma variável independente, elas podem ser separadas pelo símbolo de adição (+). No código abaixo, na primeira linha, temos quatro variáveis usadas para prever “class”. Na segunda linha, o ponto indica que todas as variáveis serão empregadas para prever “class”.

```
> formula=class~Sepal.Length+Sepal.Width+Petal.Length+Petal.Width
> formula=class~.
```

Estrutura de Objetos

Agora, iremos estudar classes que podem armazenar conjunto de da- dos em diversas estruturas e formas. As principais classes estão na tabela se- guinte.

Classe	Descrição
Vetores	Conjunto simples de valores do mesmo tipo
Matrizes	Conjunto de linhas e colunas, onde cada coluna deve ter o mesmo tipo de dado
Arrays	Arrays podem ter mais de duas dimensões
Listas	Permite agrupar listas de diferentes objetos que podem ter inclusive diferentes tipos e tamanhos
Data Frames	Semelhante a uma tabela de um banco de dados
Séries Temporais	Estrutura para séries temporais
Fatores	Estrutura para variáveis categóricas

Conjuntos de dados nativos do R

A instalação do R contém vários conjuntos de dados clássicos que podem ser usados em testes e estudos no R. Você pode obter uma lista dos conjuntos de dados nativos do R através da função “data”.

```
> data()

Data sets in package ‘datasets’:

BJsales                             Sales Data with Leading Indicator
BJsales.lead (BJsales)              Sales Data with Leading Indicator
BOD                                 Biochemical Oxygen Demand
DNase                               Elisa assay of DNase
Formaldehyde                        Determination of Formaldehyde
Indometh                            Pharmacokinetics of Indomethacin
InsectSprays                        Effectiveness of Insect Sprays
LakeHuron                           Level of Lake Huron 1875-1972
Loblolly                            Growth of Loblolly pine trees
Nile                                Flow of the River Nile
Orange                              Growth of Orange Trees
OrchardSprays                       Potency of Orchard Sprays
Theoph                              Pharmacokinetics of Theophylline
UCBAdmissions                       Student Admissions at UC Berkeley
UKgas                               UK Quarterly Gas Consumption
USArrests                           Violent Crime Rates by US State
USPersonalExpenditure               Personal Expenditure Data
VADeaths                            Death Rates in Virginia (1940)
WWWusage                            Internet Usage per Minute
WorldPhones                         The World's Telephones
ability.cov                         Ability and Intelligence Tests
```

Esta lista anterior é parcial e a saída se dá em nova janela. Para visualizar um conjunto de dados, basta digitar o nome do objeto no console.

```
> USArrests
               Murder Assault UrbanPop Rape
Alabama          13.2     236       58 21.2
Alaska           10.0     263       48 44.5
Arizona           8.1     294       80 31.0
Arkansas          8.8     190       50 19.5
California        9.0     276       91 40.6
Colorado          7.9     204       78 38.7
Connecticut       3.3     110       77 11.1
Delaware          5.9     238       72 15.8
Florida          15.4     335       80 31.9
Georgia          17.4     211       60 25.8
Hawaii            5.3      46       83 20.2
Idaho             2.6     120       54 14.2
Illinois         10.4     249       83 24.0
Indiana           7.2     113       65 21.0
Iowa              2.2      56       57 11.3
Kansas            6.0     115       66 18.0
Kentucky          9.7     109       52 16.3
Louisiana        15.4     249       66 22.2
Maine             2.1      83       51  7.8
Maryland         11.3     300       67 27.8
Massachusetts     4.4     149       85 16.3
Michigan         12.1     255       74 35.1
Minnesota         2.7      72       66 14.9
Mississippi      16.1     259       44 17.1
Missouri          9.0     178       70 28.2
Montana           6.0     109       53 16.4
Nebraska          4.3     102       62 16.5
Nevada           12.2     252       81 46.0
New Hampshire     2.1      57       56  9.5
New Jersey        7.4     159       89 18.8
New Mexico       11.4     285       70 32.1
New York         11.1     254       86 26.1
North Carolina   13.0     337       45 16.1
North Dakota      0.8      45       44  7.3
Ohio              7.3     120       75 21.4
Oklahoma          6.6     151       68 20.0
Oregon            4.9     159       67 29.3
Pennsylvania      6.3     106       72 14.9
Rhode Island      3.4     174       87  8.3
South Carolina   14.4     279       48 22.5
South Dakota      3.8      86       45 12.8
Tennessee        13.2     188       59 26.9
Texas            12.7     201       80 25.5
Utah              3.2     120       80 22.9
Vermont           2.2      48       32 11.2
Virginia          8.5     156       63 20.7
Washington        4.0     145       73 26.2
West Virginia     5.7      81       39  9.3
Wisconsin         2.6      53       66 10.8
Wyoming           6.8     161       60 15.6
```

Existem dados em pacotes que podem não estar carregados. Para se obter dados de um pacote não carregado, basta carregá-lo e em seguida digitar o nome do conjunto de dados.

```
> library("chemometrics", lib.loc="~/R/win-library/3.5")
> data(cereal)
```

Para se obter informações do conjunto de dados, utilize a função “help”.

```
> help(cereal)
```

Vetores

Um vetor armazena um conjunto de dados do mesmo tipo. A atribuição de valores a um vetor ocorre através da sintaxe c(), onde o conjunto de dados é listado e separado por vírgula. A seguir, são criados três vetores de tipos diferentes:

```
> # Criação de Vetor Numérico
> Num<-c(2,4,6,8,10)
> # Criação de Vetor de Caracteres
> Char<-c("a","e","i","o","u")
> # Criação de Vetor Booleano
> Bool<-c(T,F,T,F,T)
> Num
[1]  2  4  6  8 10
> Char
[1] "a" "e" "i" "o" "u"
> Bool
[1]  TRUE FALSE  TRUE FALSE  TRUE
```

Manipulação de vetores

Ao digitar o nome do objeto, o R lista todos os valores do vetor. Caso você deseje, um subconjunto de dados, use como regra geral o emprego de colchetes logo após o nome do objeto para que seja especificado o filtro mostrando o resultado desejado. Para se obter o valor da terceira posição do vetor Num, digite o comando

```
> Num[3]
[1] 6
```

No exemplo seguinte, são obtidos os valores das posições 1, 3 e 5 do vetor Char.

```
> Char[c(1,3,5)]
[1] "a" "i" "u"
```

Pode-se ainda obter-se um intervalo de valores, de acordo com a posição dos elementos contidos no vetor.

```
> Char[c(2:4)]
[1] "e" "i" "o"
```

Opcionalmente, pode-se nos interessar apenas os valores dos elementos superiores a um determinado valor. Por exemplo, para se obter os valores superiores a 5 existentes no vetor Num, digite

```
> Num[Num > 5]
[1]  6  8 10
```

Pode-se saber o número de elementos no vetor use a função “length”, para se obter os valores máximo e mínimo, empregue as funções “max” e “min”, para se obter a soma dos elementos de um vetor, utilize a função “sum”.

```
> length(Num)
[1] 5
> max(Num)
[1] 10
> min(Num)
[1] 2
> sum(Num)
[1] 30
```

Para se fazer uma ordenação decrescente dos valores de um vetor, empregue o argumento “decreasing”, definindo o mesmo como verdadeiro. O seu valor padrão é falso.

```
> sort(Num,decreasing=TRUE)
[1] 10  8  6  4  2
```

Adição de vetores

Caso você disponha de dois vetores de mesmo tamanho e quiser soma os valores de cada posição, basta somar os dois vetores. Observe o código abaixo

```
> Num_1=c(2,4,6,8,10)
> Num_2=c(1,3,5,7,9)
> Num_1+Num_2
[1]  3  7 11 15 19
```

A omissão de valores não disponíveis em operações matemáticas de modo que o resultado não seja NA, pode ser feita usando o argumento “na.rm”. Por exemplo, cria-se um vetor numérico onde uma posição é NA, em seguida executa-se uma função para calcular o desvio padrão “sd”. O resultado esperado é NA. Finalmente, executa-se a mesma função, porém define-se na.rm como verdadeiro. Em consequência, a função omite os valores não disponíveis.

```
> Num<-c(10,15,20,NA)
```

```
> Num
[1] 10 15 20 NA
> sd(Num)
[1] NA
> sd(Num,na.rm = TRUE)
[1] 5
```

Matrizes e Arrays

A matriz é uma estrutura de dados com duas dimensões: linhas e colunas. Arrays são matrizes que podem conter mais de duas dimensões, sendo estas ilimitadas. A matriz comporta apenas um tipo de dado. Por exemplo, usando uma matriz nativa do R.

```
> USPersonalExpenditure
                      1940   1945  1950 1955  1960
Food and Tobacco    22.200 44.500 59.60 73.2 86.80
Household Operation 10.500 15.500 29.00 36.5 46.20
Medical and Health   3.530  5.760  9.71 14.0 21.10
Personal Care        1.040  1.980  2.45  3.4  5.40
Private Education    0.341  0.974  1.80  2.6  3.64
```

Você pode acessar linhas ou colunas através de seu índice usando a notação “matriz[linha,coluna]”, onde deve-se informar o índice da linha e da coluna, respectivamente. Para saber o valor da primeira linha e terceira coluna, digitamos o comando:

```
> USPersonalExpenditure[1,3]
[1] 59.6
```

Caso queira visualizar a primeira e segunda linha e apenas a primeira e terceira coluna, use a notação “c” que empregamos para referência a valores:

```
> USPersonalExpenditure[1:2,c(1,3)]
                    1940 1950
Food and Tobacco    22.2 59.6
Household Operation 10.5 29.0
```

Ou ainda definir as linhas ou colunas

```
> USPersonalExpenditure[1:2,1:3]
                    1940 1945 1950
Food and Tobacco    22.2 44.5 59.6
Household Operation 10.5 15.5 29.0
```

Informando-se somente a linha e omitindo-se um valor de índice para coluna, é listada a primeira linha com todas as colunas

```
> USPersonalExpenditure[1,]
1940 1945 1950 1955 1960
22.2 44.5 59.6 73.2 86.8
```

Pode-se ainda acessar linhas e colunas pelos seus nomes:

```
> USPersonalExpenditure["Food and Tobacco","1940"]
[1] 22.2
```

Criação de uma matriz

Para se criar uma matriz, use-se a função “matrix”, devendo-se fornecer o número de linhas, o número de colunas e se o preenchimento da matriz será por linhas ou colunas. Por exemplo, para se criar a matriz A, digite o comando:

```
> A<-matrix(c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=TRUE)
> A
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

Definindo-se byrow como falso, muda-se a ordem de preenchimento por colunas ao invés de linhas.

```
> A<-matrix(c(1,2,3,4,5,6),nrow=2,ncol=3,byrow=FALSE)
> A
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

Esta matriz não tem nomes nas linhas e colunas. A função “dimnames” permite a adição de nomes ao objeto.

```
> dimnames(A)=list(c("Linha1","Linha2"),c("Coluna1","Coluna2","Coluna3
"))
> A
       Coluna1 Coluna2 Coluna3
Linha1       1       3       5
Linha2       2       4       6
```

Pode-se ainda fazer a substituição de um valor no conjunto de dados. Por exemplo, iremos atribuir o valor 55 para o elemento da primeira linha e segunda coluna.

```
> USPersonalExpenditure
                      1940   1945  1950 1955  1960
Food and Tobacco    22.200 44.500 59.60 73.2 86.80
Household Operation 10.500 15.500 29.00 36.5 46.20
Medical and Health   3.530  5.760  9.71 14.0 21.10
Personal Care        1.040  1.980  2.45  3.4  5.40
Private Education    0.341  0.974  1.80  2.6  3.64
> newUSP<-USPersonalExpenditure
> newUSP[1,2]<-55
> newUSP
                      1940   1945  1950 1955  1960
Food and Tobacco    22.200 55.000 59.60 73.2 86.80
Household Operation 10.500 15.500 29.00 36.5 46.20
Medical and Health   3.530  5.760  9.71 14.0 21.10
```

```
Personal Care        1.040  1.980  2.45  3.4  5.40
Private Education    0.341  0.974  1.80  2.6  3.64
```

Para se saber as dimensões de uma matriz, use a função “dim”.

```
> dim(USPersonalExpenditure)
[1] 5 5
```

Listas

A lista pode conter diversos objetos de estruturas diferentes, em posições diferentes da lista. Pode-se dizer que uma lista comporta um vetor de objetos. Por exemplo, uma lista pode conter um vetor, uma matriz e até outra lista. Iremos usar a lista “ability.cov” existente no R.

```
> ability.cov
$`cov`
        general picture  blocks   maze reading   vocab
general  24.641   5.991  33.520  6.023  20.755  29.701
picture   5.991   6.700  18.137  1.782   4.936   7.204
blocks   33.520  18.137 149.831 19.424  31.430  50.753
maze      6.023   1.782  19.424 12.711   4.757   9.075
reading  20.755   4.936  31.430  4.757  52.604  66.762
vocab    29.701   7.204  50.753  9.075  66.762 135.292

$center
[1] 0 0 0 0 0 0

$n.obs
[1] 112
```

Esta lista é composta de três elementos: cov, center e n.obs. Por exemplo, para acessar os elementos de center, usaremos o comando abaixo:

```
> ability.cov$center
[1] 0 0 0 0 0 0
```

No código seguinte, confirma-se que os elementos que compõe de uma lista podem ser de classes diferentes. Usando-se a função “class”, pode-se observar que cov é uma matriz e center é um vetor numérico.

```
> class(ability.cov$center)
[1] "numeric"
> class(ability.cov$cov)
[1] "matrix"
```

O acesso ao elemento de uma matriz contida em uma lista pode ser feito através do índice da linha e coluna. Por exemplo,

```
> ability.cov$cov[1,3]
[1] 33.52
```

No entanto, você pode acessar os elementos do vetor diretamente pelo índice do vetor e do elemento. O índice do elemento do vetor deverá estar entre dois colchetes. Enquanto que, o índice do elemento estará de acordo com a estrutura deste. Por exemplo, se for um data frame devemos informar linha e coluna, se for um vetor, apenas a posição no índice, entre outros.

```
> ability.cov[[1]][2,1]
[1] 5.991
> ability.cov[[3]][1]
[1] 112
```

Data Frame

Um data frame é a classe que mais se assemelha a uma tabela em um banco de dados. Diferente da matriz, um data frame suporta colunas com tipos de dados diferentes similar à uma tabela em um banco de dados. Considere o data frame “animals” existente no R.

```
> USPersonalExpenditure
                      1940   1945  1950 1955  1960
Food and Tobacco    22.200 44.500 59.60 73.2 86.80
Household Operation 10.500 15.500 29.00 36.5 46.20
Medical and Health   3.530  5.760  9.71 14.0 21.10
Personal Care        1.040  1.980  2.45  3.4  5.40
Private Education    0.341  0.974  1.80  2.6  3.64
> USPersonalExpenditure[5,5]
[1] 3.64
> USPersonalExpenditure["Private Education","1960"]
[1] 3.64
```

Para se criar um data frame, usamos a função “data.frame”. No exemplo, criamos três vetores, cada um com três elementos. Finalmente, criamos o data.frame “pessoas” que recebe como parâmetro os três vetores e tem como nome das colunas, o nome dos vetores.

```
> índice=c(1,2,3)
> Nomes=c("Dalva", "Marta", "Deborah")
> Brasileiro=c(TRUE,TRUE,TRUE)
> pessoas=data.frame(índice,Nomes,Brasileiro)
> pessoas
  índice   Nomes Brasileiro
1      1   Dalva       TRUE
2      2   Marta       TRUE
3      3 Deborah       TRUE
```

Podemos verificar se o R inferiu corretamente os tipos já na criação dos vetores.

```
> class(pessoas$índice)
[1] "numeric"
> class(pessoas$Nomes)
```

```
[1] "factor"
> class(pessoas$Brasileiro)
[1] "logical"
```

Funções Attach, Detach e With

Caso você tenha que fazer várias operações sobre o data frame “cars”, ficará um pouco cansativo ter que citar o nome do conjunto de dados mais a coluna ou linha. Para isso, uma alternativa interessante é usar a função “attach”. Esta função coloca o conjunto de dados no caminho de busca do R, de forma que você possa acessar colunas, simplesmente, pelo nome da coluna.

No código abaixo, observaremos alguns registros de “cars”. Tentaremos, em seguida, somar a coluna velocidade usando apenas o nome da coluna.

```
> head(cars)
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
> sum(speed)
Error: object 'speed' not found
```

Usaremos, então, a função attach com o argumento cars, depois repetimos a soma

```
> attach(cars)
> sum(speed)
[1] 770
```

Finalmente, removemos o data frame do conjunto de buscas do R usando a função “detach”.

```
> detach(cars)
```

A função “with” tem efeito similar, porém ela não adiciona nada no caminho do R, a facilidade é apenas no momento da chamada da função. Ao chamar a função with, deve-se informar o objeto e a operação a ser executada.

```
> with(cars,sum(speed))
[1] 770
```

Criação de Sequências

Uma forma simples de se criar uma sequência ou mesmo selecionar intervalos numéricos é usando dois pontos (:) como operador. Observe o exemplo

```
> var<-c(1:10)
> var
 [1]  1  2  3  4  5  6  7  8  9 10
```

Outra forma mais elaborada de criar uma sequência é com a função “seq”, onde além dos valores inicial e final, podemos definir os incrementos.

```
> seq(from=10,to=100,by=10)
 [1]  10  20  30  40  50  60  70  80  90 100
> seq(from=1,to=10,by=0.5)
 [1]  1.0  1.5  2.0  2.5  3.0  3.5  4.0  4.5  5.0  5.5  6.0  6.5  7.0
7.5  8.0  8.5  9.0  9.5 10.0
```

Comprimento

A função “length” também já foi estudada. Ela retorna o tamanho de uma estrutura em número de posições. Ela também pode ser usada para alterar o tamanho de uma estrutura qualquer. No exemplo seguinte, criamos dois vetores e verificamos o comprimento de cada um. Em seguida, definimos que o vetor varA deve ser do mesmo comprimento de varB. Neste caso, como uma nova posição foi adicionada, ela é preenchida com NA.

```
> varA<-c(10,15,20,25)
> varB<-c(10,15,20,25,30)
> length(varA);length(varB)
[1] 4
[1] 5
> length(varA)<-length(varB)
> length(varA)
[1] 5
> varA
[1] 10 15 20 25 NA
```

Conversões de Tipos

O R possui uma série de funções e formas de conversão de valores. De uma forma geral, você pode usar a sintaxe “as.[class]” para converter um tipo. Considere o exemplo a seguir, criamos um objeto do tipo caractere para depois convertê-lo em numérico.

```
> var<-"9"
> class(var)
[1] "character"
> var1<-as.numeric(var)
> class(var1)
[1] "numeric"
```

A lista de possíveis conversões é grande:

as.numeric()

as.character()

as.vector()

as.matrix()

as.data.frame()

as.factor()

Funções Head e Tail

Um ambiente de linha de comando não é muito amigável para a visualização de dados em forma de tabela. O R possui várias funções que permitem a visualização dos primeiros ou últimos registros de um objeto.

```
> head(DNase)
  Run       conc density
1   1 0.04882812   0.017
2   1 0.04882812   0.018
3   1 0.19531250   0.121
4   1 0.19531250   0.124
5   1 0.39062500   0.206
6   1 0.39062500   0.215
> tail(DNase)
    Run   conc density
171  11  3.125   0.994
172  11  3.125   0.980
173  11  6.250   1.421
174  11  6.250   1.385
175  11 12.500   1.715
176  11 12.500   1.721
```

Função summary

A função summary faz um resumo de um objeto. O resumo é apresentado por coluna: valor Mínimo, 1º. Quartil, Mediana, Média, 3º. Quantil, valor Máximo. Por exemplo

```
> summary(DNase)
      Run          conc             density
 10     :16   Min.   : 0.04883   Min.   :0.0110
 11     :16   1st Qu.: 0.34180   1st Qu.:0.1978
 9      :16   Median : 1.17188   Median :0.5265
 1      :16   Mean   : 3.10669   Mean   :0.7192
 4      :16   3rd Qu.: 3.90625   3rd Qu.:1.1705
 8      :16   Max.   :12.50000   Max.   :2.0030
 (Other):80
```

Visualização de Nomes de Linhas e Colunas

O R possui as funções “rownames” e “colnames” que permitem a visualização e alteração de nomes de linhas e colunas de um conjunto de dados.

```
> colnames(USArrests)
[1] "Murder"   "Assault"  "UrbanPop" "Rape"
> rownames(USArrests)
 [1] "Alabama"        "Alaska"         "Arizona"        "Arkansas"
"California"     "Colorado"
 [7] "Connecticut"    "Delaware"       "Florida"        "Georgia"
"Hawaii"         "Idaho"
[13] "Illinois"       "Indiana"        "Iowa"           "Kansas"
"Kentucky"       "Louisiana"
[19] "Maine"          "Maryland"       "Massachusetts"  "Michigan"
"Minnesota"      "Mississippi"
[25] "Missouri"       "Montana"        "Nebraska"       "Nevada"
"New Hampshire"  "New Jersey"
[31] "New Mexico"     "New York"       "North Carolina" "North Dakota"
"Ohio"           "Oklahoma"
[37] "Oregon"         "Pennsylvania"   "Rhode Island"   "South Carolin
a" "South Dakota"   "Tennessee"
[43] "Texas"          "Utah"           "Vermont"        "Virginia"
"Washington"     "West Virginia"
[49] "Wisconsin"      "Wyoming"
```

Aplicação de Funções sobre Linhas ou Colunas

Com o R, pode-se aplicar uma função sobre todos os valores de linhas ou colunas. A função “apply” aplica a função especificada sobre um conjunto de dados definido no primeiro parâmetro. O segundo parâmetro deve ser informado como 1 para aplicar a função sobre linhas ou 2 para aplicar sobre as colunas. O terceiro parâmetro é a função. No exemplo seguinte, iremos aplicar a função “median”, inicialmente sobre a coluna, e em seguida, sobre a linha da matriz USPersonalExpenditure.

```
> apply(USPersonalExpenditure,1,median)
Food and Tobacco Household Operation Medical and Health  Personal Care Private Education
    59.60          29.00             9.71           2.45         1.80
> apply(USPersonalExpenditure,2,median)
 1940  1945  1950  1955  1960
 3.53  5.76  9.71 14.00 21.10
```

Números Aleatórios

A função “sample” números aleatórios dentro de um intervalo definido. Sua utilização básica recebe o nome de tamanho da amostra:

```
> sample(10)
 [1]  6  4  9  7  2  5  3  1  8 10
```

Podemos ainda definir um intervalo de números de onde os valores serão retirados. É possível através do argumento “replace”, do tipo booleano, definir se haverá substituição.

```
> amostra<-sample(1:1000,size=10,replace=FALSE)
> amostra
 [1]   2 742 155 861 118 977 367 780 518 164
```

Se rodarmos novamente, como é esperado em uma função para gerar números aleatórios, provavelmente os números gerados serão diferentes.

```
> amostra<-sample(1:1000,size=10,replace=FALSE)
> amostra
 [1] 870 625 934 620 770 314 531 364 591 469
```

Caso você tenha interesse em repetir o processo ou obter o mesmo resultado, utilize a função “set.seed” imediatamente antes de chamar sample. A função set.seed deve receber um argumento do tipo inteiro para garantir que os números gerados sejam sempre os mesmos.

```
> set.seed(2013)
> sample(1:1000,size=10,replace=FALSE)
 [1] 464 952 784 762 252 723 908 793 927 813
> set.seed(2013)
> sample(1:1000,size=10,replace=FALSE)
 [1] 464 952 784 762 252 723 908 793 927 813
> sample(1:1000,size=10,replace=FALSE)
 [1]   5 247 753 856 622 712 973 935  77 327
> sample(1:1000,size=10,replace=FALSE)
 [1] 550  46  75 808 271 462 156 669 804 456
```

Distribuição de Probabilidades

O R inclui a funcionalidade para operações com distribuições de probabilidades.

Para cada distribuição, existem quatro operações básicas indicadas pelas letras:

d - calcula a densidade de probabilidade de f (x) no ponto.

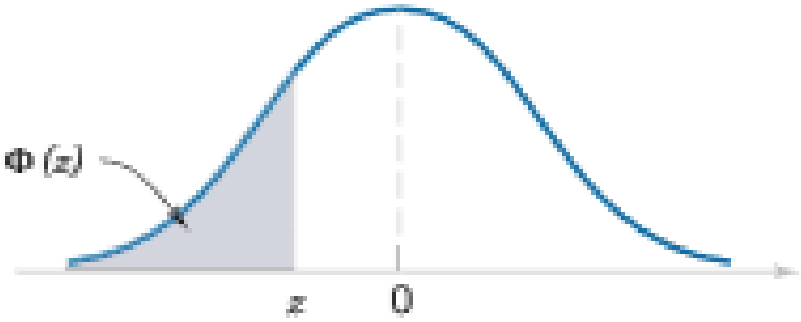
p - calcula a função de probabilidade acumulada F(x) no ponto.

q - calcula o quantil correspondente a uma dada probabilidade.

r - retira uma amostra da distribuição.

Distribuição Normal

$$\Phi(z) = P(Z \leq z) = \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} du$$



Exercícios

a) Qual o valor da probabilidade quando Z = -1,00?
```
> pnorm(-1)
[1] 0.1586553
```
b) Qual o valor da probabilidade quando Z = -1,75?
```
> pnorm(-1.75)
[1] 0.04005916
```
c) Qual o valor de Z quando a probabilidade é igual a 0,1003?
```
> qnorm(0.1003)
[1] -1.279844
```
d) Qual o valor de Z quando a probabilidade é igual a 0,0250?
```
> qnorm(0.0250)
[1] -1.959964
```

Distribuição t

Exercícios

a) Qual o valor de t quando o tamanho da amostra é igual a 100 com um nível de confiança de 95% para um teste bilateral?
```
> qt(0.025,99)
[1] -1.984217
> qt(0.975,99)
[1] 1.984217
```

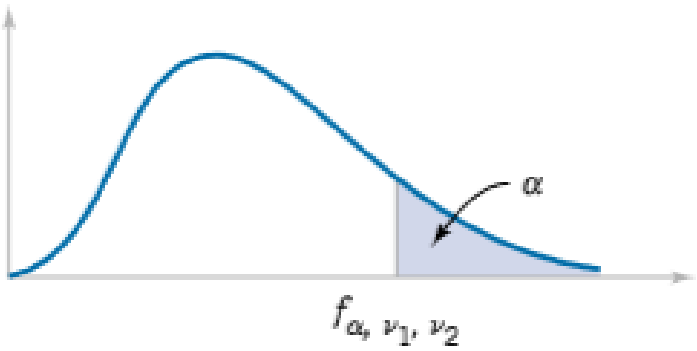
b) Qual o valor de t quando o tamanho da amostra é igual a 10 com um nível de confiança de 99% para um teste bilateral?

```
> qt(0.005,9);qt(0.995,9)
[1] -3.249836
[1] 3.249836
```

c) Qual o valor de t quando o tamanho da amostra é igual a 9 com um nível de confiança de 95% para um teste unilateral superior?

```
> qt(0.95,8)
[1] 1.859548
```

Distribuição F



Exercícios

a) Qual o valor de F quando o número de graus de liberdade do numerador é igual a 3 e o número de graus de liberdade do denominador é igual a 16 para um nível de confiança de 95%?

```
> qf(0.95,3,16)
[1] 3.238872
```

b) Qual o valor de F quando o número de graus de liberdade do numerador é igual a 3 e o número de graus de liberdade do denominador é igual a 16 para um nível de confiança de 99%?

```
> qf(0.99,3,16)
[1] 5.292214
```

Outras distribuições

dexp - densidade da distribuição exponencial.

rchisq - retira amostra de uma distribuição qui-quadrado.

qt - calcula o quantil de uma distribuição t-student.

pf - calcula f(x) de uma distribuição F.

rbeta - retira uma amostra de uma distribuição beta.

dgamma - densidade de uma distribuição gama.

rpois - retira amostra de uma distribuição Poisson.

dunif - densidade de uma distribuição uniforme (contínua).

Funções: apply, lapply, sapply, vapply, mapply, rapply, tapply, Aggregate e by

apply - Quando você quer aplicar a função às linhas ou colunas de uma matriz.

```
> M <- matrix(seq(1,16), 4, 4)
> M
     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
> apply(M, 1, min)
[1] 1 2 3 4
> apply(M, 2, max)
[1]  4  8 12 16
> M1 <- array( seq(32), dim = c(4,4,2))
> M1
, , 1

     [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16

, , 2

     [,1] [,2] [,3] [,4]
[1,]   17   21   25   29
[2,]   18   22   26   30
[3,]   19   23   27   31
[4,]   20   24   28   32

> apply(M1, 1, sum)
[1] 120 128 136 144
> apply(M1, c(1,2), sum)
     [,1] [,2] [,3] [,4]
[1,]   18   26   34   42
[2,]   20   28   36   44
[3,]   22   30   38   46
[4,]   24   32   40   48
```

lapply - Quando você quer aplicar uma função para cada elemento de uma lista e receber uma lista de volta. Este é o carro-chefe de muitas das outras funções *apply.

```
> x <- list(a = 1, b = 1:3, c = 10:100)
> lapply(x, FUN = length)
$a
[1] 1

$b
[1] 3

$c
[1] 91

> lapply(x, FUN = sum)
$a
```

```
[1] 1

$b
[1] 6

$c
[1] 5005
```

sapply - Quando você quer aplicar a função para cada elemento de uma lista, porém quer retornar um **vetor** ao invés de uma lista.

```
> x <- list(a = 1, b = 1:3, c = 10:100)
> sapply(x, FUN = length)
 a  b  c
 1  3 91
> sapply(x, FUN = sum)
   a    b    c
   1    6 5005
```

vapply - Para quando você quer utilizar o sapply, mas talvez precise de um código mais rápido.

```
> x <- list(a = 1, b = 1:3, c = 10:100)
> # Note que uma vez que o avanço aqui é principalmente a velocidade,
# este
> # Exemplo é apenas para ilustração. Estamos dizendo que R
> # Tudo voltou por length () deve ser um número inteiro de
> # Comprimento 1.
> vapply(x, FUN = length, FUN.VALUE = 0)
 a  b  c
 1  3 91
```

mapply - Para quando você tem várias estruturas de dados diferentes (e.g. vetores, listas) e você quer aplicar a função para os primeiros elementos de cada e então os segundos, etc., forçando o resultado em um vetor ou array como em sapply.

```
# Soma os 1ºs elementos, os 2ºs elementos, etc.
> mapply(sum, 1:5, 1:5, 1:5)
[1]  3  6  9 12 15

##Para fazer rep(1,4), rep(2,3), etc.
> mapply(rep, 1:4, 4:1)
[[1]]
[1] 1 1 1 1

[[2]]
[1] 2 2 2

[[3]]
[1] 3 3

[[4]]
[1] 4
```

rapply - Para quando você quer aplicar a função para cada elemento de uma **lista aninhada** de forma recursiva.

```
> myFun <- function(x){
+   if (is.character(x)){
+     return(paste(x,"!",sep=""))
+   }
+   else{
+     return(x + 1)
+   }
+ }

> l <- list(a = list(a1 = "Boo", b1 = 2, c1 = "Eeek"),
+           b = 3, c = "Yikes",
+           d = list(a2 = 1, b2 = list(a3 = "Hey", b3 = 5)))

> rapply(l,myFun)
    a.a1     a.b1     a.c1        b        c     d.a2  d.b2.a3  d.b2.b
3
  "Boo!"      "3"  "Eeek!"      "4" "Yikes!"      "2"   "Hey!"      "6
"

> rapply(l, myFun, how = "replace")
$a
$a$a1
[1] "Boo!"

$a$b1
[1] 3

$a$c1
[1] "Eeek!"


$b
[1] 4

$c
[1] "Yikes!"

$d
$d$a2
[1] 2

$d$b2
$d$b2$a3
[1] "Hey!"

$d$b2$b3
[1] 6
```

tapply - Para quando você quiser aplicar a função à **subsetores** de um vetor e estes são definidos por outro vetor.

```
# Um vetor
> x <- 1:20
# A fator (do mesmo tamanho!) definindo os grupos:
> y <- factor(rep(letters[1:5], each = 4))
# # Adicione os valores em x em cada subgrupo definido por y:
> tapply(x, y, sum)
```

```
 a  b  c  d  e
10 26 42 58 74
```

Aggregate e by - É relativamente fácil de recolher dados em R utilizando um ou mais BY variáveis e uma função definida.

```
> aggdata <-aggregate(mtcars, by=list(cyl,vs),FUN=mean, na.rm=TRUE)
> print(aggdata)
   Group.1 Group.2  mpg    cyl   disp    hp     drat       wt     qsec    vs     am      gear     carb
1       4       0 26.00000   4 120.30  91.0000 4.430000 2.140000 16.70000  0 1.0000000 5.000000 2.000000
2       6       0 20.56667   6 155.00 131.6667 3.806667 2.755000 16.32667  0 1.0000000 4.333333 4.666667
3       8       0 15.10000   8 353.10 209.2143 3.229286 3.999214 16.77214  0 0.1428571 3.285714 3.500000
4       4       1 26.73000   4 103.62  81.8000 4.035000 2.300300 19.38100  1 0.7000000 4.000000 1.500000
5       6       1 19.12500   6 204.55 115.2500 3.420000 3.388750 19.21500  1 0.0000000 3.500000 2.500000
> detach(mtcars)
```

AULA PRÁTICA 1 – APRESENTAÇÃO DE DADOS

Atividade 1: Construir uma tabela de contingência

Criação de um vetor de dados

basedados <- c(10, 30, 40,20,15,35,30,45,75)

Transformação em matriz, com três linhas, três colunas

tabcontingencia <- matrix(basedados, 3, 3, byrow=TRUE)

Para visualizar a tabela de contingência criada

```
> tabcontingencia

     [,1] [,2] [,3]
[1,]   10   30   40
[2,]   20   15   35
[3,]   30   45   75
```

Inserção de rótulos nas linhas

rownames(tabcontingencia) <- c("homem", "mulher","total")

Inserção de rótulos nas colunas

colnames(tabcontingencia) <- c("peixe", "carne vermelha","total")

Para visualizar a tabela de contingência criada após a inserção dos rótulos nas linhas e colunas

tabcontingencia

```
> tabcontingencia

       peixe carne vermelha total
homem     10             30    40
mulher    20             15    35
total     30             45    75
```

Atividade 2: Criação do gráfico de barras

Criação de um vetor de dados

basedados <- c(10, 30, 20, 15)

Transformação em matriz, com duas linhas, duas colunas, por sequência em linha

tabcontingencia <- matrix(basedados, 2, 2, byrow=TRUE)

Gráfico de barras vertical

barplot(c(tabcontingencia), col="darksalmon", lwd=2, space=2, main="Gráfico de barras de quatro variáveis")

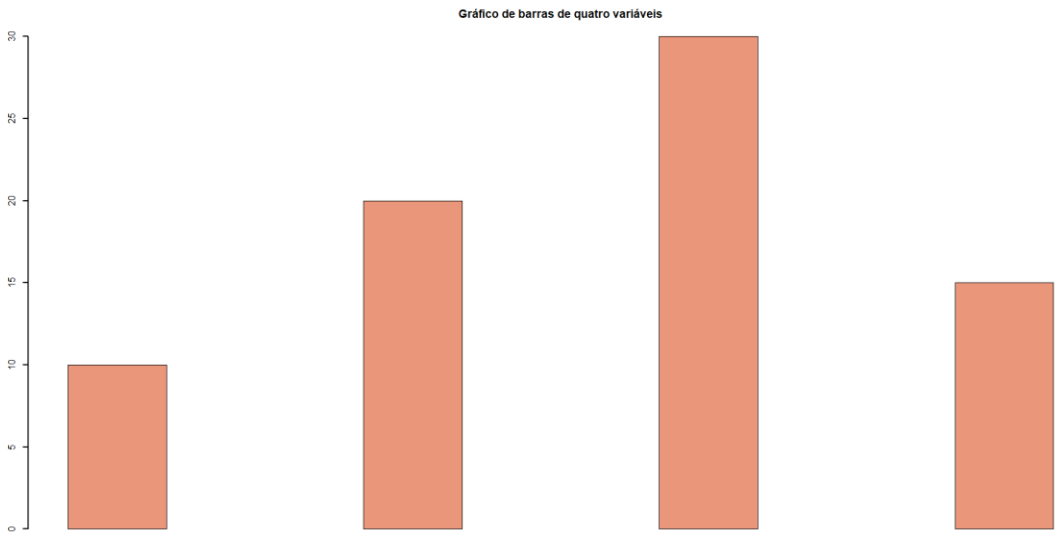
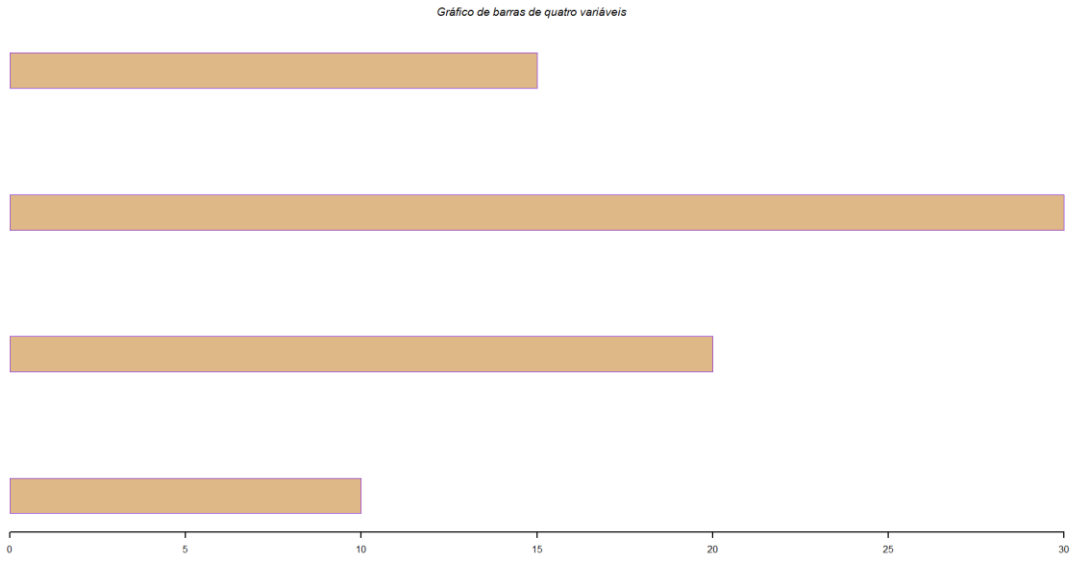


Gráfico de barras horizontal, com título em itálico.

barplot(c(tabcontingencia), col="burlywood ", lwd=2, space=3, main="Gráfico de barras de quatro variáveis", horiz=TRUE, font.main=3, border=" BlueViolet")



Atividade 3: Criação do gráfico de pizza

vetor de vendas de sorvetes, em kg.

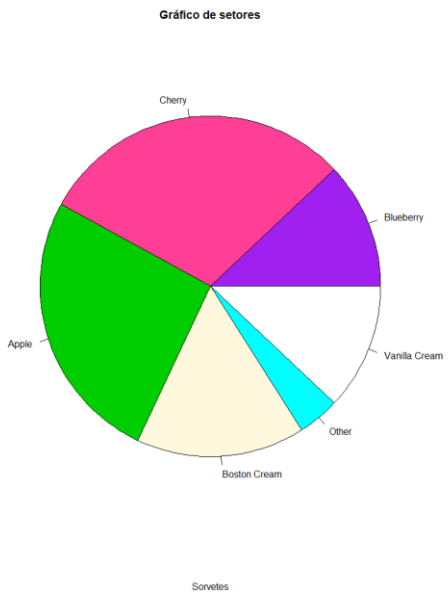
sorvete.vendas <- c(0.12, 0.3, 0.26, 0.16, 0.04, 0.12)

adicionando nomes

names(sorvete.vendas) <- c("Blueberry", "Cherry", "Apple", "Boston Cream", "Other", "Vanilla Cream")

Criação do gráfico de setores

pie(sorvete.vendas, col = c("purple", "violetred1", "green3", "cornsilk", "cyan", "white"), main="Gráfico de setores", sub="Sorvetes")



Atividade 4: Construir um gráfico de barras paralelas

Criação de um vetor de dados

risco <- c(332, 132, 16, 14, 113, 231)

Transformação em matriz, com duas linhas e três colunas

tabcont.risco <- matrix(risco, 2, 3, byrow=TRUE)

Visualização da matriz criada

```
> tabcont.risco

     [,1] [,2] [,3]
[1,]  332  132   16
[2,]   14  113  231
```

Inserção de rótulos nas linhas

rownames(tabcont.risco) <- c("Crescimento", "Valorização")

#Inserção de rótulos nas colunas

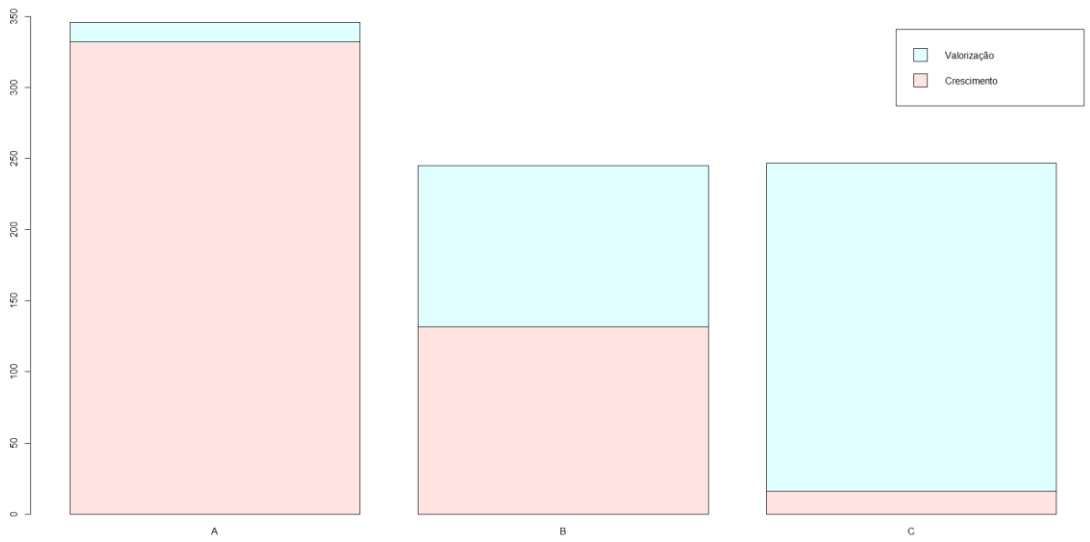
colnames(tabcont.risco) <- c("A", "B", "C")

Visualização da matriz criada após a inserção de rótulos nas linhas e colunas

```
> tabcont.risco

                A   B   C
Crescimento   332 132  16
Valorização    14 113 231
```

Criação do gráfico de barras paralelas.

barplot(tabcont.risco, beside=FALSE, col=c("mistyrose", "lightcyan", "lavender"), legend=rownames(tabcont.risco), ylim=c(0, 350))



Atividade 5: Organizando dados numéricos usando a disposição ordenada

Para reprodução dos dados

set.seed(1)

Geração de números aleatórios em uma distribuição normal (90;5,8).

Normal <- rnorm(100, mean=90, sd=5.8)

Ordena de forma crescente

```
> sort(Normal, decreasing=FALSE)

[1]  77.15474  78.46176  79.53124  81.16331  81.46964  82.01305  82.59577  82.72893
82.89725  83.44969
```

```
[11]  83.94402  84.58223  85.15335  85.24128  85.68902  85.88231  85.89653  86.00522
86.36657  86.39680

[21]  86.45025  86.58078  86.67506  86.70172  86.85338  87.22673  87.25428  87.42891
87.59303  87.71312

[31]  87.87012  88.22875  88.23573  88.53050  89.04576  89.09639  89.21596  89.21668
89.34839  89.40383

[41]  89.65598  89.67445  89.68793  89.73939  89.77241  89.90610  90.00641  90.16241
90.43118  90.43248

[51]  90.88887  91.06513  91.09500  91.54917  91.69039  91.91115  91.93111  91.97849
92.11458  92.14611

[61]  92.24850  92.26109  92.30901  92.42406  92.75796  92.82709  93.22865  93.23922
93.30437  93.33953

[71]  93.44463  93.44489  93.54221  93.59499  94.00049  94.04239  94.06124  94.28228
94.42642  94.45749

[81]  94.53639  94.76308  95.11042  95.33007  95.47425  96.16598  96.38015  96.52460
96.73034  96.83290

[91]  97.00563  97.88034  98.31154  98.50022  98.76833  99.20363  99.25263 101.48632
102.60115 103.92938
```

Ordena de forma decrescente

sort(Normal, decreasing=TRUE)

Atividade 6: Construir um gráfico de ramos-e-folhas

Criação do vetor de dados

dados<-c(40,80,42,44,49,60,90,20,35)

Criação do gráfico de ramo-e-folhas

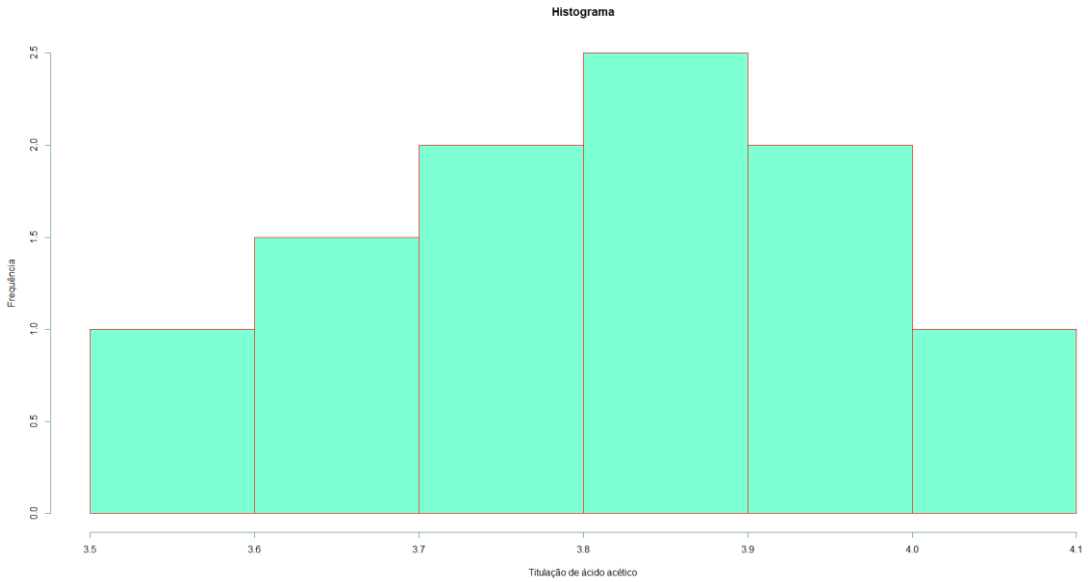
```
> stem(dados, scale=2, width=80, atom=1e-08)

       The decimal point is 1 digit(s) to the right of the |

         2 | 0
         3 | 5
         4 | 0249
         5 |
         6 | 0
         7 |
         8 | 0
         9 | 0
> stem(dados, scale=1, width=80, atom=1e-08)

       The decimal point is 1 digit(s) to the right of the |

         2 | 05
         4 | 0249
         6 | 0
         8 | 00
```

Atividade 7: Construir um Histograma

Criação do vetor titulação

titulacao <- c(3.91, 4.01, 3.61, 3.83, 3.75, 3.91, 3.82, 3.70, 3.50, 3.77, 3.96, 3.85, 3.67, 3.83, 3.77, 3.51, 3.85, 4.04, 3.74, 3.97)

hist(titulacao, main="Histograma", xlab="Titulação de ácido acético", ylab="Frequência", col="aquamarine", border="red", fg="cadetblue4", prob=TRUE)



Atividade 8: Gráfico de dispersão dos dados

Criação do vetor de dados

Calorias <- c(240, 260, 350, 350, 420, 510, 530)

Criação do vetor de dados

Gordura <- c(8.0, 3.5, 22.0, 20.0, 16.0, 22.0, 19.0)

Construção do gráfico de dispersão

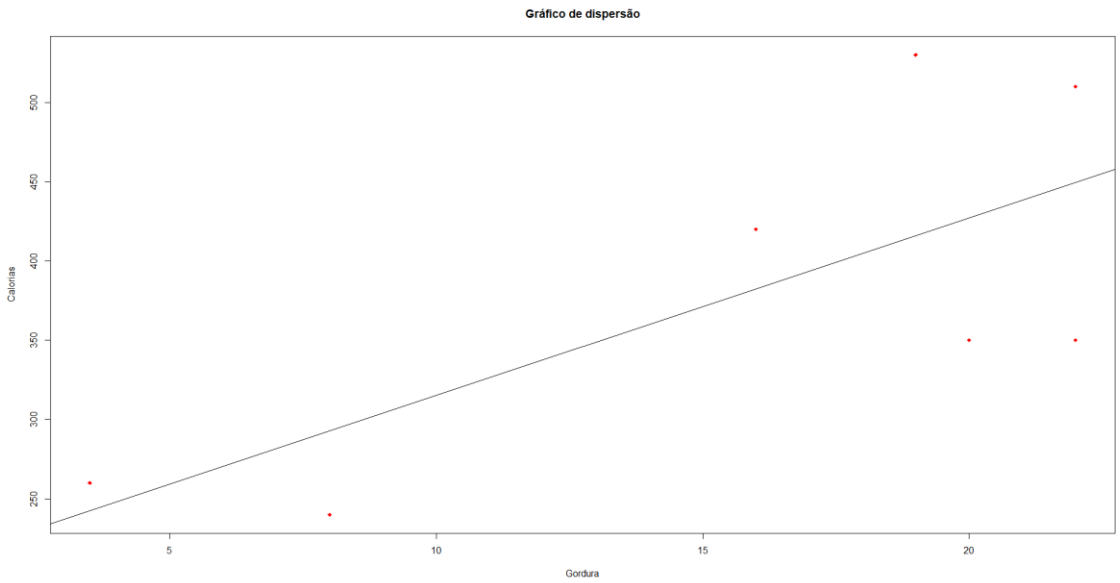
plot(Calorias ~ Gordura, col= "red", pch= 18, main="Gráfico de dispersão", xlab= "Gordura", ylab= "Calorias")

Modelo de regressão por mínimos quadrados

m <- lm(Calorias ~ Gordura)

Reta de regressão por mínimos quadrados

abline(m)



Sumário do modelo de regressão

```
> summary(m)

Call:

        lm(formula = Calorias ~ Gordura)

        Residuals:
             1      2      3      4      5      6      7
        -52.79  17.62 -99.61 -77.21  37.60  60.39 114.00

        Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
        (Intercept)  203.180     82.958   2.449   0.0580 .
        Gordura       11.201      4.834   2.317   0.0683 .
        ---
        Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

        Residual standard error: 86.06 on 5 degrees of freedom
        Multiple R-squared:  0.5178,   Adjusted R-squared:  0.4213
        F-statistic: 5.368 on 1 and 5 DF,  p-value: 0.06831
```

Coeficiente de correlação de Pearson

```
> cor(Calorias, Gordura)

[1] 0.7195601
```

Covariância

```
> cov(Gordura, Calorias)

[1] 591.6667
```

AULA PRÁTICA 2 – MEDIDAS DESCRITIVAS

Atividade 1: Estatística Descritiva

a) Calcule a média aritmética, a mediana, o primeiro e o terceiro quartil.

b) Calcule a variância, o desvio padrão, a amplitude, a amplitude interquartil, o coeficiente de variação e os escores Z.

Criação do vetor tempo

tempo <- c(9.66, 5.90, 8.02, 5.79, 8.73, 3.82, 8.01, 8.35, 10.49, 6.68, 5.64, 4.08, 6.17, 9.91, 5.47)

Resumo dos cinco números

```
> summary(tempo)
   Min.  1st Qu.  Median    Mean  3rd Qu. Max.
  3.820   5.715   6.680   7.115   8.540  10.490
```

```
# Variância
> var(tempo)
[1] 4.335512
> # Desvio padrão
> sd(tempo)
[1] 2.082189
> # Amplitude
> max(tempo) - min(tempo)
[1] 6.67
> # Amplitude interquartil
> IQR(tempo)
[1] 2.825
```

Coeficiente de variação

tempo.rsd <- (sd(tempo)/mean(tempo))*100

Visualização do coeficiente de variação

```
> tempo.rsd
```

```
[1] 29.26615
```

Padronização dos dados

z <- scale(tempo)

Mostra os valores dos dados padronizados

```
> z
```

```
           [,1]
 [1,]  1.2224313
 [2,] -0.5833603
 [3,]  0.4347988
 [4,] -0.6361893
 [5,]  0.7757860
 [6,] -1.5823089
 [7,]  0.4299961
 [8,]  0.5932858
 [9,]  1.6210502
[10,] -0.2087546
```

```
[11,] -0.7082289
[12,] -1.4574403
[13,] -0.4536891
[14,]  1.3424972
[15,] -0.7898737
```

Atividade 02: Construir um Gráfico de Caixas

Gerar um mesmo conjunto de números aleatórios

set.seed(1)

Geração de 100 números com distribuição normal, média igual a 1 e desvio padrão igual a 0,2

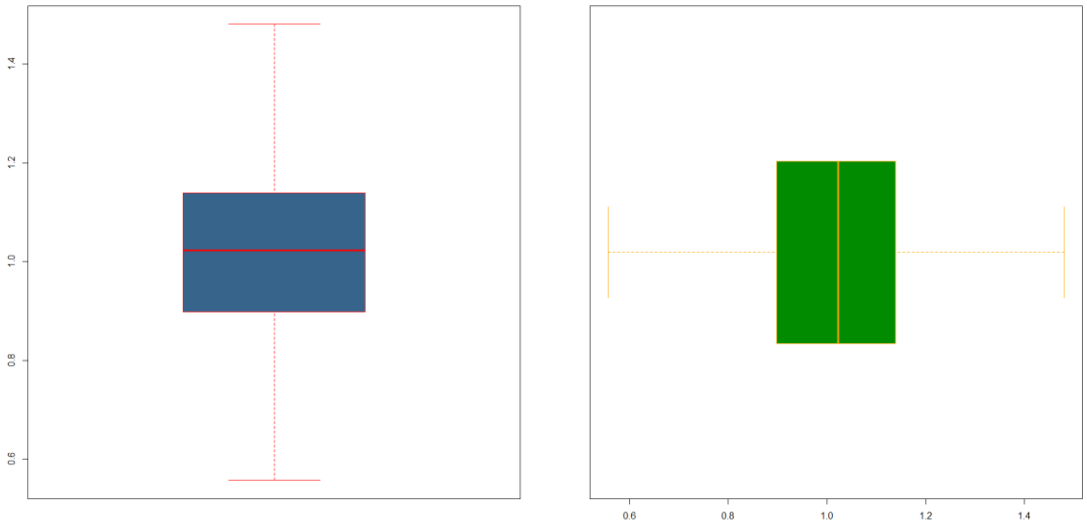
caixa <- rnorm(100, mean=1, sd=0.2)

Colocar os dois gráficos na mesma janela

par(mfcol=c(1,2));

boxplot(caixa, col="steelblue4", border="red");

boxplot(caixa, col="green4", border="orange", horizontal=TRUE)



AULA PRÁTICA 3 – TESTE PARA DETECÇÃO DE OUTLIERS

Atividade 01: Teste de Grubbs para detecção de valores atípicos

Fazer download do pacote estatístico

install.packages("outliers")

Habilitar o pacote outliers

library(outliers)

Criar o vetor de dados

sample<-c(0.189, 0.167, 0.187, 0.183, 0.186, 0.182, 0.181, 0.184, 0.177)

Executar o teste estatístico

type = 10: teste o menor valor

Executar o teste estatístico

grubbs.test(sample, type=10,opposite=FALSE, two.sided = TRUE)

```
Grubbs test for one outlier
data:  sample
G = 2.24850, U = 0.28903, p-value = 0.03868
alternative hypothesis: lowest value 0.167 is an outlier
```

type = 11: testa os dois valores extremos baseado na razão da variação do desvio padrão

grubbs.test(sample, type=11,opposite=FALSE, two.sided = TRUE)

```
Grubbs test for two opposite outliers
data:  sample
G = 3.34740, U = 0.19348, p-value = 0.3531
alternative hypothesis: 0.167 and 0.189 are outliers
```

Atividade 02: Teste de Dixon para detecção de valores atípicos

Habilitar o pacote outliers

library(outliers)

Criar o vetor de dados

sample<-c(0.189, 0.167, 0.187, 0.183, 0.186, 0.182, 0.181, 0.184, 0.177)

Executar o teste estatístico para verificar se o menor valor é atípico

dixon.test(sample)

```
Dixon test for outliers

data:  sample

Q = 0.5, p-value = 0.1137

alternative hypothesis: lowest value 0.167 is an outlier
```

Executar o teste estatístico para verificar se os valores extremos são atípicos

dixon.test(sample, opposite=TRUE)

```
Dixon test for outliers

data:  sample

Q = 0.16667, p-value = 0.8924

alternative hypothesis: highest value 0.189 is an outlier
```

AULA PRÁTICA 4 – DISTRIBUIÇÃO NORMAL

Atividade 1: Distribuição Normal Padronizada

Determine a probabilidade de X = 9 quando a média populacional é igual a 7 e o desvio padrão populacional é igual a dois.

a)P(X=9), μ= 7,0 segundos e s = 2,0 segundos

Área acumulada sob a Curva Normal – probabilidade

```
> pnorm(9, 7, 2, log=FALSE)
[1] 0.8413447
```

Determine a probabilidade de X > 9 quando a média populacional é igual a 7 e o desvio padrão populacional é igual a dois.

b)P(X>9), μ= 7,0 segundos e s = 2,0 segundos

Área acumulada sob a Curva Normal – probabilidade

```
> 1 - pnorm(9, 7, 2, log = FALSE)
[1] 0.1586553
```

Ou

```
> pnorm(9, 7, 2, lower.tail = FALSE)
[1] 0.1586553
```

Determine a probabilidade de 7 < X < 9 quando a média populacional é igual a 7 e o desvio padrão populacional é igual a dois

c)P(7<X<9), μ= 7,0 segundos e s = 2,0 segundos

Área acumulada sob a Curva Normal – probabilidade

```
> pnorm(9, 7, 2) - pnorm(7, 7, 2)
[1] 0.3413447
```

d) Construa o Gráfico de probabilidade normal

Gráfico de probabilidade normal

custo.energia <- c(96, 171, 202, 178, 147, 102, 153, 197, 127, 82, 157, 185, 90, 116, 172, 111, 148, 213, 130, 165, 141, 149, 206, 175, 123, 128, 144, 168, 109, 167, 95, 163, 150, 154, 130, 143, 187, 166, 139, 149, 108, 119, 183, 151, 114, 135, 191, 137, 129, 158) #criação do vetor custo

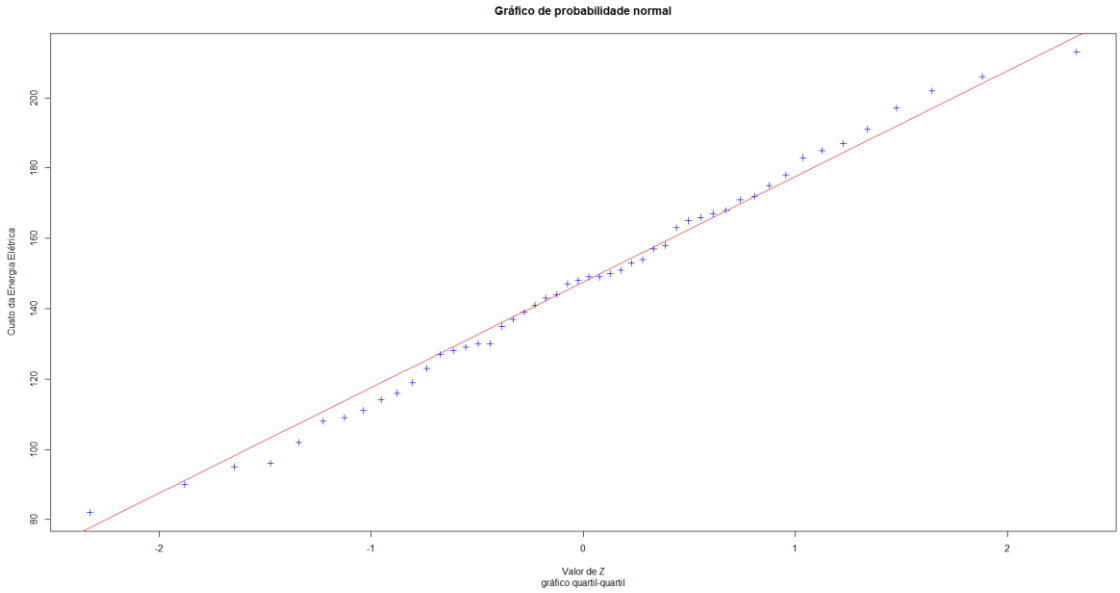
par(mfrow=c(1,1))

Criação do gráfico de probabilidade normal

qqnorm(custo.energia, main="Gráfico de probabilidade normal", sub="gráfico quartil-quartil", xlab="Valor de Z", ylab="Custo da Energia Elétrica", pch=3, col="blue")

Inserção da linha de probabilidade normal

qqline(custo.energia, col="red")



Atividade 02: Construção do gráfico de distribuição normal

Os valores 1 e 20 são os valores mínimo e máximo respectivamente da sequência de dados

x <- seq(1,20,length=1000)

Distribuição normal considerando a média 12 e desvio padrão igual a 2

y <- dnorm(x,mean=12, sd=2)

Distribuição normal considerando a média 10 e desvio padrão igual a 3

y1 <- dnorm(x,mean=10, sd=3)

Distribuição normal considerando a média 11 e desvio padrão igual a 3

y2 <- dnorm(x,mean=11, sd=3)

Construção do primeiro gráfico de distribuição normal

plot(x,y,type= "l", lwd=3, col="black", main="Distribuição Normal")

Inserção do segundo gráfico de distribuição normal representado por uma linha pontilhada ao invés de linha cheia

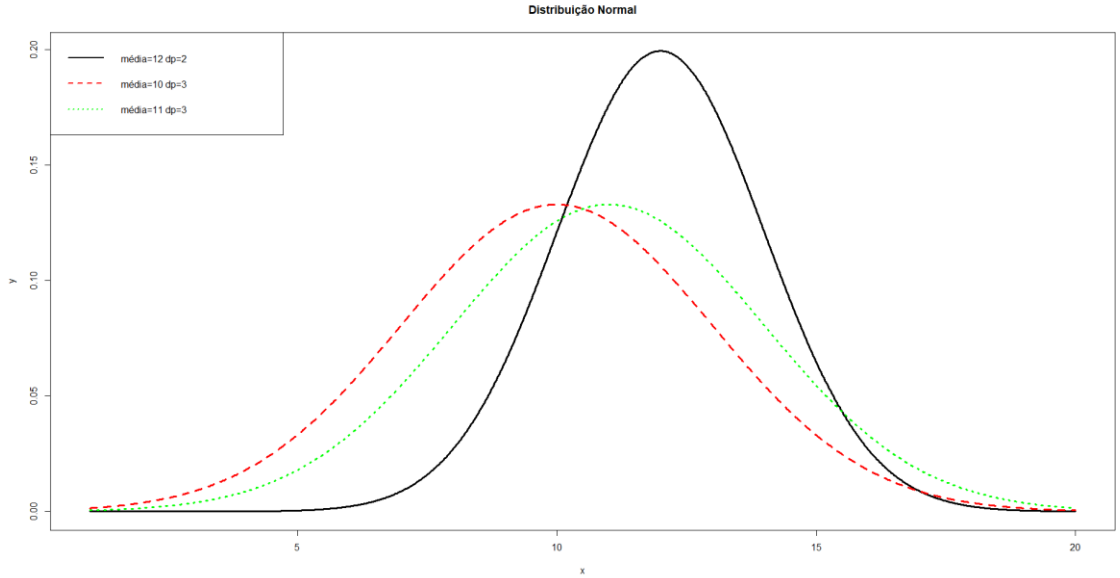
lines(x,y1, lty=2, col="red", lwd=3)

Inserção do terceiro gráfico de distribuição normal representado por uma linha pontilhada ao invés de linha cheia

lines(x,y2, lty=3, col="green", lwd=3)

Coloca os diferentes gráficos de distribuição normal com cores diferentes

legend("topleft", c("média=12 dp=2", "média=10 dp=3", "média=11 dp=3"), lty=1:3, col=c("black", "red", "green"), lwd=2)



AULA PRÁTICA 5 – INTERVALO DE CONFIANÇA

Atividade 1: Estimativa do Intervalo de Confiança para a Média Aritmética (desvio padrão conhecido):

a) Construa uma estimativa para o intervalo de confiança de 99% da média aritmética da população relativa à quantidade de tinta contida em uma lata de 1 galão.

```
# Insere o valor da média previamente fornecido

media <- 0.995

# Insere o valor do desvio padrão previamente fornecido

desv.pad <- 0.02

# Insere o tamanho da amostra previamente fornecido

n <- 50

# erro.padrao <- qnorm(0.995)*s/sqrt(n)

erro.padrao <- qnorm(0.995)*desv.pad /sqrt(n)

# Cálculo do limite inferior

limite.inferior <- media - erro.padrao

# Cálculo do limite superior

limite.superior <- media + erro.padrao

# Exibe os valores de limite inferior e superior

limite.inferior; limite.superior
```

```
[1] 0.9877145
[1] 1.002286
```

b) Construa uma estimativa para o intervalo de confiança de 95%.

```
# Insere o valor da média previamente fornecido

media <- 0.995

# Insere o valor do desvio padrão previamente fornecido

desv.pad <- 0.02

# Insere o tamanho da amostra previamente fornecido

n <- 50

# Cálculo do erro padrão

erro.padrao2 <- qnorm(0.975)* desv.pad /sqrt(n)
```

Cálculo do limite inferior

limite.inferior2 <- media - erro.padrao2

Cálculo do limite superior

limite.superior2 <- media + erro.padrao2

Exibe os valores dos limites inferior e superior

limite.inferior2; limite.superior2

```
[1] 0.9894564
```

```
[1] 1.000544
```

Atividade 2: Estimativa do Intervalo de Confiança para a Média Aritmética (desvio padrão desconhecido):

Criação do vetor de dados

frango.gordura <- c(7, 8, 4, 5, 16, 20, 20, 24, 19, 30, 23, 30, 25, 19, 29, 29, 30, 30, 40, 56)

Teste t de Student

t.test(frango.gordura)

```
        One Sample t-test

        data:  frango.gordura
        t = 8.3762, df = 19, p-value = 8.413e-08
        alternative hypothesis: true mean is not equal to 0
        95 percent confidence interval:
         17.40282 28.99718
        sample estimates:
        mean of x
             23.2
```

Atividade 3: Determinar o tamanho da amostra para estimar a média aritmética

a) De que tamanho precisa ser a amostra para que a média seja detectada com erro de 50?

Valor crítico de Z superior

Zsup <- qnorm(.975)

Zsup

```
[1] 1.959964
```

```
# Valor crítico de Z inferior

Zinf<-qnorm(.025)

Zinf

[1] -1.959964

# Desvio padrão populacional

Sigma <- 400;

# Valor do erro

Erro <- 50

# Cálculo do tamanho da amostra

N <- (Zsup^2)*(Sigma^2)/(Erro^2)

# Exibe o valor do tamanho da amostra

N

[1] 245.8534

# b) Se a administração deseja estar correta dentro dos limites de 25, que tamanho de amostra
é necessário?

# Valor de Z crítico superior

Z <- qnorm(0.975);

# Valor do desvio padrão

Sigma <- 400;

# Valor do erro

Erro2 <- 25

# Cálculo do tamanho da amostra

N2 <- (Z^2)*(Sigma^2)/(Erro2^2)

# Exibe o tamanho da amostra

N2

[1] 983.4135
```

AULA PRÁTICA 6 – TESTE DE HIPÓTESE PARA UMA AMOSTRA

 # Atividade 1: Utilizando o teste Z para a média aritmética com desvio padrão conhecido

```
# Exemplo 1
# Hipótese nula: média ser igual a 70
# Hipótese alternativa: média ser diferente de 70
# Nível de confiança: 95%
# Teste bilateral ou bicaudal
# Média aritmética = 69,1
# Desvio padrão populacional = 3,5
# Tamanho da amostra = 49

# Resolução
# Valor de Z superior crítico
z.sup <- qnorm(0.975)
# Valor de Z inferior crítico
z.inf <- qnorm(0.025)
# Valor de Z calculado
zcalc <- (69.1-70) / (3.5 / sqrt(49) )
# Cálculo do p-valor
p.valor <- 2*(1-pnorm(abs(zcalc)))
# Mostra os valores de Z crítico, Z calculado e o p–valor.
z.sup; z.inf; zcalc; p.valor
[1] 1.959964
[1] -1.959964
[1] -1.8
[1] 0.07186064
```

OBS.: Para teste unilateral superior e inferior:

p-valor do teste unilateral superior

p.valor <- 1-pnorm(zcalc)

```
> p.valor
[1] 0.9640697
```

p-valor do teste unilateral inferior

p.valor <- pnorm(zcalc)

```
> p.valor
[1] 0.03593032
```

 # Atividade 2: Utilizando o teste t para a média aritmética com desvio padrão desconhecido.

Letra A: Teste bilateral ou bicaudal

Criação do vetor de dados

x<-c(59.3, 14.2, 32.9, 69.1, 23.1, 79.3, 51.9, 39.2, 41.8)

Testar se a média é igual a 40 (Hipótese nula)

t.test(x,mu=40,alternative="two.sided",conf.level=0.95)

```
        One Sample t-test

        data:  x
        t = 0.79558, df = 8, p-value = 0.4492
        alternative hypothesis: true mean is not equal to 40
        95 percent confidence interval:
         29.28381 62.00508
        sample estimates:
        mean of x
         45.64444
```

#Conclusão:

#1 O p-valor é 0,4492 é superior ao valor crítico, o que não permite rejeitar a hipótese nula.

#2 A função também fornece o intervalo de confiança com 95% (29,28 - 62,01) e média = 45,64.

#3 Como o valor 40 está contido no intervalo de confiança, não podemos rejeitar a hipótese nula

#LETRA B: Teste unilateral ou unicaudal superior

Testar se a média é maior que 30 (Hipótese alternativa)

t.test(x,mu=30,alternative="greater",conf.level=0.95)

```
One Sample t-test

data:  x
t = 2.2051, df = 8, p-value = 0.02927
alternative hypothesis: true mean is greater than 30
95 percent confidence interval:
 32.45133      Inf
sample estimates:
mean of x
 45.64444
```

#LETRA C: Teste unilateral ou unicaudal inferior

Testar se a média é menor que 30 (Hipótese alternativa)

t.test(x,mu=30,alternative="less",conf.level=0.95)

```
One Sample t-test

data:  x
t = 2.2051, df = 8, p-value = 0.9707
alternative hypothesis: true mean is less than 30
95 percent confidence interval:
     -Inf 58.83756
sample estimates:
mean of x
 45.64444
```

AULA PRÁTICA 7 – TESTE DE HIPÓTESE PARA DUAS AMOSTRAS

Atividade 1: Utilizando o teste Z para populações independentes

```
# Criação do vetor de dados
maq1 <- c(16.03, 16.04, 16.05, 16.05, 16.02, 16.01, 15.96, 15.98, 16.02, 15.99)
# Criação do vetor de dados
maq2 <- c(16.02, 15.97, 15.96, 16.01, 15.99, 16.03, 16.04, 16.02, 16.01, 16.00)
# Diferença entre as médias
D <- (mean(maq1)-mean(maq2))
# Desvio padrão da 1ª amostra
dp1 <- 0.020
# Desvio padrão da 2ª amostra
dp2 <- 0.025
# Tamanho de cada amostra. Como são iguais, será representado apenas por n
n <- 10
# Valor de Z crítico superior
zsup <- qnorm(0.975)
# Valor de Z crítico inferior
zinf <- qnorm(0.025)
# Valor de Z calculado
zcalc <-((D) - 0)/sqrt(((dp1^2)/n)+((dp2^2)/n))
# cálculo do p - valor
p.valor <- 2*(1-pnorm(abs(zcalc)))
# Mostra os valores de Z crítico superior e inferior e Z calculado e o p – valor.
zsup; zinf; zcalc; p.valor
[1] 1.959964
[1] -1.959964
[1] 0.9877296
[1] 0.3232851
```

Atividade 2: Utilizando o teste t (populações independentes)

Letra A: Teste bilateral ou bicaudal: (Hipótese nula: as médias são iguais)

Criar os vetores x e y

x<-c(0.795, 0.864, 0.841, 0.683, 0.777, 0.720)

y<-c(0.765, 0.735, 1.003, 0.778, 0.647, 0.740, 0.612)

Executar o teste t

t.test(x,y,alternative="two.sided",var.equal=TRUE)

```
Two Sample t-test

data:  x and y
t = 0.44459, df = 11, p-value = 0.6652
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.1015879  0.1530164
sample estimates:
mean of x mean of y
0.7800000 0.7542857
```

Letra B: Teste unilateral ou unicaudal superior

Criar os vetores x e y

x<-c(0.795, 0.864, 0.841, 0.683, 0.777, 0.720)

y<-c(0.765, 0.735, 1.003, 0.778, 0.647, 0.740, 0.612)

Executar o teste t

t.test(x,y,alternative="greater",var.equal=TRUE)

```
Two Sample t-test

data:  x and y
t = 0.44459, df = 11, p-value = 0.3326
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
 -0.07815739         Inf
sample estimates:
mean of x mean of y
0.7800000 0.7542857
```

#Letra C: Teste unilateral ou unicaudal inferior

Criar os vetores x e y

x<-c(0.795, 0.864, 0.841, 0.683, 0.777, 0.720)

y<-c(0.765, 0.735, 1.003, 0.778, 0.647, 0.740, 0.612)

Executar o teste t

t.test(x,y,alternative="less",var.equal=TRUE)

```
Two Sample t-test

data:  x and y
t = 0.44459, df = 11, p-value = 0.6674
alternative hypothesis: true difference in means is less than 0
95 percent confidence interval:
     -Inf 0.129586
sample estimates:
mean of x mean of y
0.7800000 0.7542857
```

Atividade 3: Teste t pareado (populações dependentes)

Letra A: Teste bilateral ou bicaudal

Testar se as médias são iguais (Hipótese nula)

Vetores de dados

initial_value<-c(16, 20, 21, 22, 23, 22, 27, 25, 27, 28)

final_value<-c(19, 22, 24, 24, 25, 25, 26, 26, 28, 32)

Executar o teste t pareado

t.test(final_value,initial_value,alternative="two.sided",paired=TRUE)

```
Paired t-test

data:  final_value and initial_value
t = 4.4721, df = 9, p-value = 0.00155
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 0.9883326 3.0116674
sample estimates:
mean of the differences
                      2
```

Letra B: Teste unilateral ou unicaudal superior

t.test(final_value,initial_value,alternative="greater",paired=TRUE)

```
Paired t-test

data:  final_value and initial_value
t = 4.4721, df = 9, p-value = 0.0007749
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
 1.180207      Inf
sample estimates:
mean of the differences
                      2
```

Letra C: Teste unilateral ou unicaudal inferior

t.test(final_value,initial_value,alternative="less",paired=TRUE)

```
        Paired t-test

        data:  final_value and initial_value
        t = 4.4721, df = 9, p-value = 0.9992
        alternative hypothesis: true difference in means is less than 0
        95 percent confidence interval:
             -Inf 2.819793
        sample estimates:
        mean of the differences
                              2
```

AULA PRÁTICA 8 – ANOVA

Atividade 1: Utilizando o teste ANOVA de fator único

#hipótese nula: as médias são iguais.

#hipótese alternativa: pelo menos uma das médias diferentes

#nível de confiança: 95%

Criação do vetor de dados

dados <-c(39.5, -31.0, 39.0, -1.1, -20.7, 31.9, -4.5, -45.0, 14.1, -8.0, -73.3, 5.4)

Criação do rótulo dos dados

rotulo <- factor(rep(c("Especialista", "Leitores", "Dardos"),1:4,12),ordered=TRUE)

resultado <- aov(dados~rotulo)

```
> resultado

        Call:
           aov(formula = dados ~ rotulo)

        Terms:
                          rotulo Residuals
        Sum of Squares  9195.435  3763.927
        Deg. of Freedom        2         9

        Residual standard error: 20.45029
        Estimated effects are balanced

> summary(resultado)
                     Df Sum Sq Mean Sq F value  Pr(>F)
        rotulo        2   9195    4598   10.99 0.00383 **
        Residuals     9   3764     418
```

Atividade 2: Teste de Tukey para comparações múltiplas

Resolução

dados <-c(39.5, -31.0, 39.0, -1.1, -20.7, 31.9, -4.5, -45.0, 14.1, -8.0, -73.3, 5.4)

rotulo <- factor(rep(c("Especialista", "Leitores", "Dardos"),1:4,12),ordered=TRUE)

resultado <- aov(dados~rotulo)

```
> resultado

       Call:
          aov(formula = dados ~ rotulo)

       Terms:
                         rotulo Residuals
       Sum of Squares  9195.435  3763.927
       Deg. of Freedom        2         9

       Residual standard error: 20.45029
       Estimated effects are balanced

> summary(resultado)
                    Df Sum Sq Mean Sq F value  Pr(>F)
       rotulo        2   9195    4598   10.99 0.00383 **
       Residuals     9   3764     418
```

Intervalo de amplitude crítica Qs, com prob, gl numerador e gl denominador

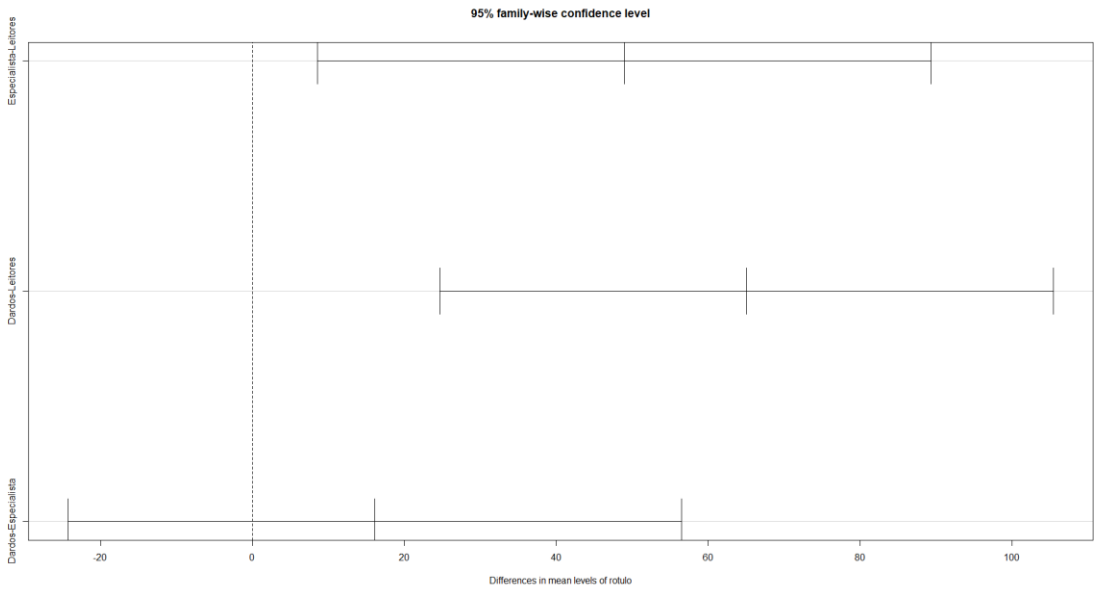
```
> qtukey(.95, 2, 9)

[1] 3.199173
```

Teste de comparações múltiplas HSD de Tukey

tcm <- TukeyHSD(resultado, "rotulo", ordered = TRUE, conf.level=0.95)

```
> tcm

         Tukey multiple comparisons of means
           95% family-wise confidence level
           factor levels have been ordered

       Fit: aov(formula = dados ~ rotulo)

       $rotulo
                                diff        lwr      upr     p adj
       Especialista-Leitores 48.975   8.601104  89.3489 0.0198472
       Dardos-Leitores       65.100  24.726104 105.4739 0.0038035
       Dardos-Especialista   16.125 -24.248896  56.4989 0.5291573
```

plot(tcm)



Atividade 3: Utilizando o teste de Levene para homogeneidade da variância

Hipótese nula: as variâncias são iguais.

Hipótese alternativa: pelo menos uma das variâncias diferentes

Nível de confiança: 95%

Resolução

Criação do data frame dos dados

tratamento <- data.frame(rotulo=rotulo, dados=dados)

Exibe tratamento

```
> tratamento

          rotulo dados
1  Especialista  39.5
2      Leitores -31.0
3        Dardos  39.0
4  Especialista  -1.1
5      Leitores -20.7
6        Dardos  31.9
7  Especialista  -4.5
8      Leitores -45.0
9        Dardos  14.1
10 Especialista  -8.0
11     Leitores -73.3
12       Dardos   5.4
```

install.packages(“car”)

library (car)

Teste de Levene modificado (diferença absoluta da mediana)

leveneTest(dados, rotulo)

```
Levene's Test for Homogeneity of Variance (center = median)
      Df F value Pr(>F)
group  2  0.1011 0.9049
       9
```

Teste de Levene original (diferença absoluta da média)

leveneTest(dados, rotulo, center=mean)

```
Levene's Test for Homogeneity of Variance (center = mean)
      Df F value Pr(>F)
group  2   0.185 0.8342
       9
```

AULA PRÁTICA 9 – REGRESSÃO LINEAR SIMPLES

Atividade 1: Regressão por mínimos quadrados

Calcule a equação do modelo

Construa o gráfico de dispersão (y versus x)

x: sangue e y: pressão sonora

Criação do vetor de dados

sangue<-c(1, 0, 1, 2, 5, 1, 4, 6, 2, 3, 5, 4, 6, 8, 4, 5, 7, 9, 7, 6)

Criação vetor de dados

sonora<-c(60, 63, 65, 70, 70, 70, 80, 80, 80, 80, 85, 89, 90, 90, 90, 90, 94, 100, 100, 100)

Regressão linear, y em função de x

modelo <- lm(sonora~sangue)

summary(modelo)

```
Call:
lm(formula = sonora ~ sangue)

Residuals:
     Min       1Q   Median       3Q      Max 
-15.2093  -2.9517   0.1345   5.2126  10.6345 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  64.4283     3.0910  20.844 4.71e-14 ***
sangue        4.1562     0.6219   6.683 2.87e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 6.931 on 18 degrees of freedom
Multiple R-squared:  0.7127,	Adjusted R-squared:  0.6968 
F-statistic: 44.66 on 1 and 18 DF,  p-value: 2.872e-06
```

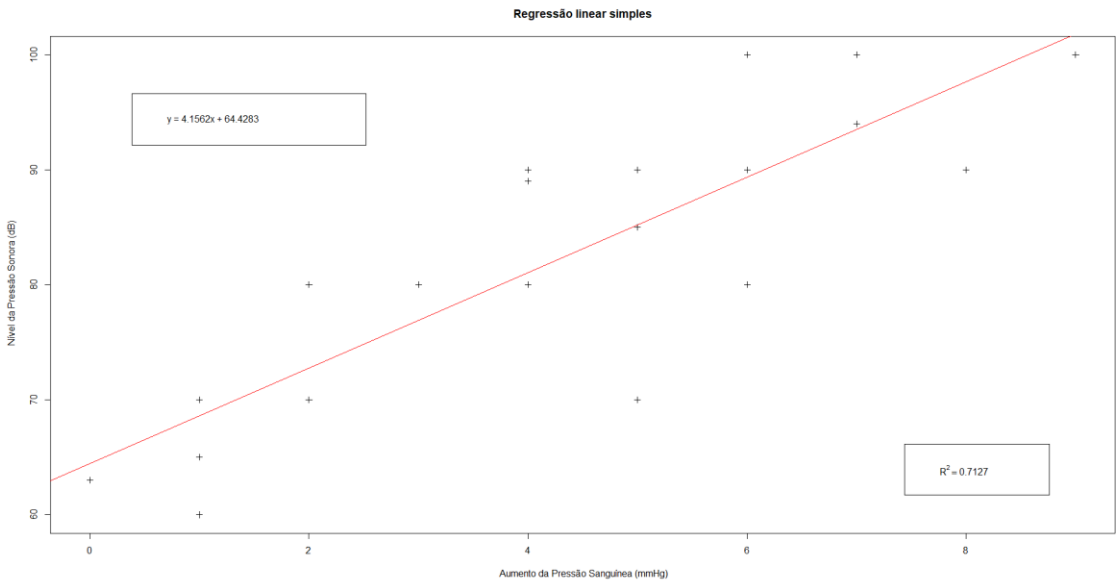
plot(sangue, sonora, ylab=" Nível da Pressão Sonora (dB)", xlab=" Aumento da Pressão Sanguínea (mmHg)", main="Regressão linear simples", pch=3)

Inserção da reta de mínimos quadrados, com os coeficientes angular e linear do modelo calculado.

abline(modelo, col="red")

legend(locator(1), cex=1.0, expression(paste(R^{"2"}) == 0.7127))

legend(locator(1), cex=1.0, expression(paste("y = 4.1562x + 64.4283")))



Calcule os resíduos e construa o gráfico de resíduos (resíduos vs x)

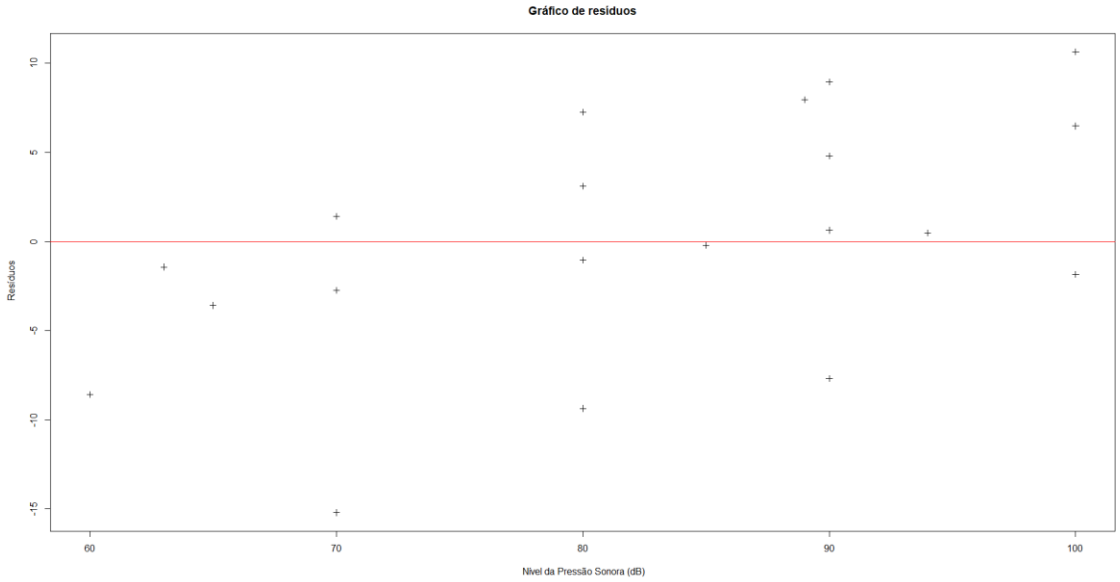
Previsão dos dados pelo método dos mínimos quadrados

preditos <- predict(modelo)

Cálculo dos resíduos

resíduos<-c(sonora-preditos)

Construção do gráfico de resíduos

plot(sonora, resíduos, xlab=" Nível da Pressão Sonora (dB)", ylab=" Resíduos", main="Gráfico de resíduos", pch=3)

abline(h=0,col="red")